“Automated CI/CD DevOps Pipeline for Cloud Native Application”

A

Project Report

Submitted in partial fulfillment of the requirement for the

award ofDegree of

Bachelor of Computer Applications

Submitted to

452 - Cimage College, Patna

Submitted By:
MD AYAN
Registration No: 202345200041
BCA - 3rd Year
Session-2023-2026

Under Joint Guidance of
Prof. Raju Upadhyay &
Prof. Sanjeev Kumar
[Asst. Professor]
Computer Science &
Applications

Cimage College, Patna

Department of Computer Science & Applications

Session 2023-26

DATE:……

CERTIFICATE

This is to certify that the work embodies in this project entitled, **"Automated CI/CD DevOps Pipeline for Cloud Native Application",** being submitted by **MD Ayan (Registration Number: 202345200041)** in partial fulfillment of the requirement for the award of "**Bachelor of Computer Applications"** to ***PATLIPUTRA UNIVERSITY, PATNA*** during the academic year 2022-25 is a record of bonafide piece of work, carried out by him/her under our/my supervision and guidance in the **"Department of Computer Science & Applications", Cimage College, Patna.**

Dr. Neeraj Agrawal Director	**Dr. Amit Kumar Shukla** Head of Department	**Prof. Nitish Kumar Rohatgi** Principal

DATE:……

APPROVAL CERTIFICATE

The project report entitled **"Automated CI/CD DevOps Pipeline for Cloud Native Application",** being submitted by **MD Ayan (Registration Number: 202345200041)** has been examined by us and is hereby approved for the award of degree "**Bachelor of Computer Applications",** for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, the opinion expressed or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner) (External Examiner)

Date: Date:

Cimage College, Patna

Department of Computer Science & Applications
Session 2023-2026

Date:.....

DECLARATION

I, Md Ayan hereby declare that the work, which is being presented in the project report entitled **"Automated CI/CD DevOps Pipeline for Cloud Native Application",** partial fulfillment of the requirements for the award of degree of "**Bachelor of Computer Applications**" submitted in the department of **Computer Science & Applications (Cimage College)** is an authentic record of my own work carried under the joint guidance of **Prof. Raju Upadhyay Sir and Prof. Sanjeev Kumar Sir.** I have not submitted the matter embodied in this report for the award of any other degree.

Md Ayan

Registration No : 202345200041

BCA 3rd Year

Session-2023-2026

ACKNOWLEDGEMENT

"A journey is easier when you travel together. Interdependence is certainly more valuable than independence."

I would like to thank **Prof. Raju Upadhyay Sir** and **Prof. Sanjeev Kumar Sir** for providing regular guidance and insight into my project work. I also thank them for all the advice he has givenme in the past year, and for always having time for me, whenever I needed.

I give special thanks to **Dr. Amit Kumar Shukla Sir, Prof. Ravi Kumar Soni Sir,** and **Prof. Anjesh Kumar Sir** for always being willing to help find solutions to any problems I had with my work.

"The completion of any project depends upon the cooperation, coordination, and combined efforts of several resources of knowledge, inspiration, and energy".

I also extend my deepest gratitude to **Director**, CIMAGE GROUP OF INSTITUITIONS, PATNA, **Dr. Neeraj Agrawal Sir, Dean** CIMAGE GROUP OF INSTITUITIONS, PATNA, **Dr. Neeraj Kumar Sir** and **Centre Head Mrs. Megha Agrawal Ma'am** for providing all the necessary facilities and true encouraging environment to bring out the best in my endeavors.

I express my gratitude and thanks to all the staff members of Computer Science departments for their sincere cooperation in furnishing relevant information to complete this Project well in time successfully.

I extend a special word to my friends, who have been a constant source of inspiration throughout my project work.

Last but not least, I must express my cordial thanks to my parents and family members who gave me moral support without the fact that it is impossible to complete my project work. With this note, I thank everyone for their support.

Name: Md Ayan

Registration No : 202345200041

BCA 3rd Year

Session-2023-2026

ABSTRACT

This report documents the design, implementation, and evaluation of a project titled "Automated CI/CD DevOps Pipeline for Cloud Native Application." The project was undertaken as the final-year dissertation requirement for the Bachelor of Computer Applications (BCA) programme at Cimage College, Patna, affiliated to Aryabhatta Knowledge University, Patna.

The core problem addressed by this project is the inefficiency of manual software deployment processes — a reality that still exists in many development teams. When developers must manually build, test, scan, package, and deploy applications, the process is slow, inconsistent, and prone to human error. A single missed step in production deployment can cause hours of downtime. This project was built to solve exactly that problem.

The solution is a fully automated CI/CD pipeline deployed on Amazon Web Services (AWS) EC2 instances. The pipeline integrates Jenkins as the central orchestration engine, Apache Maven for building the Java application, SonarQube for static code quality analysis, Trivy for security vulnerability scanning (at both the filesystem and container image levels), Nexus Repository Manager for artifact storage, Docker for containerization, and Kubernetes (bootstrapped via kubeadm) for container orchestration and deployment.

The monitoring layer — comprising Prometheus for metrics collection, Grafana for dashboard visualization, Node Exporter for system-level metrics, and Blackbox Exporter for endpoint availability probing — provides real-time operational insight into both the pipeline and the deployed application.

During implementation, I encountered and resolved several practical challenges including Kubernetes cluster bootstrapping issues, Jenkins-SonarQube webhook configuration, Docker socket permission management, and Nexus registry HTTPS configuration. These challenges and their solutions are documented in detail in the Challenges chapter.

The completed pipeline achieves a full execution time — from code commit to verified Kubernetes deployment — of under 15 minutes. All quality and security gates are enforced automatically. The results confirm that the DevOps automation approach delivers measurable improvements in deployment speed, code quality assurance, and operational visibility compared to manual processes.

Keywords:
CI/CD Pipeline, DevOps, Jenkins, Docker, Kubernetes, SonarQube, Trivy, Nexus Repository, Prometheus, Grafana, AWS EC2, Continuous Integration, Continuous Deployment, Container Orchestration, Security Scanning, Cloud Native.

TABLE OF CONTENTS

Certificate i
Approval Certificate ii
Declaration iii
Acknowledgement iv
Abstract v
Table of Contents vi
List of Figures viii
List of Tables ix
Chapter 1: Introduction 1
1.1 Background and Context 1
1.2 What is DevOps? 3
1.3 Continuous Integration (CI) 4
1.4 Continuous Deployment (CD) 6
1.5 Need for This Project 7
1.6 Project Overview 8
1.7 Scope of the Project 9
Chapter 2: Objectives 11
2.1 Primary Objectives 11
2.2 Secondary Objectives 13
2.3 Expected Outcomes 14
Chapter 3: Project Category and Advantages 15
Chapter 4: Tools and Technologies 19
4.1 Jenkins 19
4.2 Apache Maven 21
4.3 SonarQube 22
4.4 Trivy 24
4.5 Nexus Repository 25
4.6 Docker 26
4.7 Kubernetes 27
4.8 Prometheus 29
4.9 Grafana 30
4.10 AWS EC2 31
Chapter 5: Hardware and Software Requirements 33
Chapter 6: Problem Definition 38
Chapter 7: Requirement Specification 42
Chapter 8: Project Planning and Scheduling 47

Chapter 9: System Analysis and Feasibility Study **51**
Chapter 10: System Architecture **56**
Chapter 11: Implementation **61**
11.1 Phase 1 – Infrastructure Setup 61
11.2 Phase 2 – Tool Installation 66
11.3 Phase 3 – Kubernetes Cluster Setup 72
11.4 Phase 4 – Jenkins Pipeline Development 78
11.5 Phase 5 – Monitoring Setup 90
Chapter 12: Security Mechanisms **95**
Chapter 13: Testing **101**
Chapter 14: Results and Discussion **107**
Chapter 15: Challenges Faced **113**
Chapter 16: Limitations **117**
Chapter 17: Future Enhancements **119**
Chapter 18: Conclusion **123**
References **126**
Appendix **129**

LIST OF FIGURES

Figure 1: AWS EC2 Dashboard – All Project Instances Running 63
Figure 2: SSH Connection to Jenkins Server via Terminal 64
Figure 3: Java Installation Verification on Jenkins Server 65
Figure 4: Jenkins Initial Setup Screen – Unlock Jenkins 67
Figure 5: Jenkins Dashboard After Plugin Installation 68
Figure 6: Jenkins Global Tool Configuration – JDK and Maven 69
Figure 7: SonarQube Server Running – Login Dashboard 70
Figure 8: Nexus Repository Manager – Home Dashboard 71
Figure 9: Kubernetes kubeadm init – Control Plane Initialized 73
Figure 10: Kubernetes Nodes – All Nodes in Ready State 74
Figure 11: Calico CNI Plugin Installation – Pod Network Applied 75
Figure 12: Jenkins Pipeline – New Item Creation 79
Figure 13: Jenkins Credentials Manager – Configured Secrets 80
Figure 14: Jenkins Pipeline Stage View – All 13 Stages 81
Figure 15: Git Checkout Stage – Source Code Cloned 82
Figure 16: Maven Build Stage – Compilation Success 83
Figure 17: Trivy File System Scan – Vulnerability Report 84
Figure 18: SonarQube Analysis – Code Issues Dashboard 85
Figure 19: SonarQube Quality Gate – PASSED Status 86
Figure 20: Nexus Repository – Uploaded JAR Artifact 87
Figure 21: Docker Build Stage – Image Layers Created 88
Figure 22: Trivy Image Scan – Container Security Report 89
Figure 23: Docker Hub – Tagged Image Successfully Pushed 89
Figure 24: Kubernetes Deployment – Pods in Running State 90
Figure 25: kubectl get services – NodePort Service Exposed 91
Figure 26: Prometheus Web UI – All Scrape Targets UP 92
Figure 27: Grafana – Node Exporter Full System Dashboard 93
Figure 28: Grafana – Blackbox Exporter Endpoint Probe Dashboard 94
Figure 29: Jenkins Blue Ocean – Successful Full Pipeline Run 108
Figure 30: Application Accessible via Browser – NodePort 110

LIST OF TABLES

Table 1: Tool Comparison – CI/CD Orchestration Alternatives 20
Table 2: EC2 Instance Specifications per Component 33
Table 3: Port Configuration and AWS Security Group Rules 36
Table 4: Software Version Summary 37
Table 5: Functional Requirements 42
Table 6: Non-Functional Requirements – Performance 44
Table 7: Non-Functional Requirements – Security 45
Table 8: Project Phase Schedule – Gantt Summary 48
Table 9: Feasibility Study Summary 55
Table 10: SonarQube Quality Gate Conditions 70
Table 11: Pipeline Stage Test Results 102
Table 12: Performance Testing Metrics 104
Table 13: Failure Scenario Test Results 105
Table 14: Pipeline Execution Results Summary 108
Table 15: Challenges and Solutions Summary 116
Table 16: Future Enhancement Roadmap 122

CHAPTER 1: INTRODUCTION

1.1 Background and Context

Before I started working on this project, I spent some time understanding how software actually gets delivered in the real world — not just in textbook theory, but in the day-to-day reality of engineering teams. What I found was eye-opening. Even in 2024, a surprisingly large number of development teams still deploy software manually. A developer finishes writing code, then an operations engineer connects to a production server via SSH, copies a JAR file, edits a configuration, restarts a service, and hopes nothing breaks. If something does break — which happens often — the rollback is another manual process, prone to the same errors.

The history of how we got here is worth briefly examining. In the early days of software development, the Waterfall model dominated. Development happened in long phases — requirements, design, code, test, deploy — and releases might happen once or twice a year. This was slow but somewhat manageable when software was less complex and competition was less intense. As the internet grew and user expectations accelerated, development cycles shortened into Agile sprints of two to four weeks. But Agile addressed the development side, not the delivery side. You could sprint all you wanted, but getting code to production still took days or weeks of manual coordination.

DevOps emerged from this gap. The term was first coined around 2009, and it represented not just a set of tools but a fundamental cultural shift — breaking down the wall between the team that writes software (Dev) and the team that runs it (Ops). When those two groups share responsibility, share tooling, and share outcomes, the entire delivery chain becomes faster, more reliable, and more transparent.

This project grew out of my desire to understand DevOps not just in theory but by building something real. I wanted to construct a pipeline that actually automates everything — not just the build and test, but the security scanning, the quality gates,

the container image creation, the deployment to a real Kubernetes cluster, and the monitoring of everything that runs. That ambition became the basis of this report.

1.2 What is DevOps?

DevOps is not a single tool or a job title. It is a combination of cultural practices, processes, and tools that enable an organization to deliver software applications and services at high velocity — faster than those using traditional software development and infrastructure management processes. The speed enables organizations to better serve customers and compete more effectively in the market.

A useful way to think about DevOps is through the CALMS framework, which describes its five pillars:

- Culture: DevOps requires that development and operations teams share goals, communicate openly, and feel jointly responsible for the success of what they build. When a production outage happens, the question is not 'whose fault is this?' but 'how do we fix it and prevent it from happening again?'
- Automation: This is where most of the tangible value is delivered. Automating repetitive tasks — building, testing, deploying, infrastructure provisioning — means doing them faster, more consistently, and with less chance of human error.
- Lean: DevOps borrows from Lean manufacturing the principle of reducing waste and optimizing flow. Small, frequent deployments are less risky than large, infrequent ones. Identifying and eliminating bottlenecks improves throughput.
- Measurement: What gets measured gets managed. DevOps teams track metrics like deployment frequency, lead time for changes, change failure rate, and mean time to recovery. These metrics tell you whether your pipeline is actually improving delivery.

- Sharing: Knowledge hoarding slows teams down. DevOps encourages shared runbooks, post-mortems that are blameless and public, and cross-functional pairing so that everyone understands the full system.

In this project, I implemented the Automation pillar most directly — building the actual pipeline machinery. But the other pillars informed every design decision I made: why I chose to keep credentials out of the Jenkinsfile (security culture), why I built monitoring dashboards (measurement), and why I documented everything (sharing).

1.3 Continuous Integration (CI)

Continuous Integration is the practice of automatically building and testing an application every time a developer commits code to the shared repository. The key word is 'automatically' — there is no waiting for a weekly build meeting, no manually triggering a test run, no waiting for a QA engineer to pick up the ticket. The moment code lands in the repository, the pipeline fires.

Martin Fowler, who popularized the concept, described the core principle simply: 'Integrate early, integrate often.' The longer two developers work on separate branches without integrating, the more complex the eventual merge becomes. CI solves this by making integration a continuous, automated, fast-feedback event.

In my implementation, CI covers the following stages: code checkout from GitHub, compilation with Maven, unit test execution, static code analysis with SonarQube, quality gate enforcement, filesystem vulnerability scanning with Trivy, and artifact packaging. All of these happen automatically within minutes of a code push.

The practical effect is immediate: if a developer pushes code that breaks the build, the team knows within five minutes. If the code introduces a security vulnerability, Trivy flags it before the image is even built. If code quality drops below the threshold, the

Quality Gate stops the pipeline before deployment can begin. This tight feedback loop is the defining feature of effective CI.

1.4 Continuous Deployment (CD)

Continuous Deployment extends the pipeline from code validation all the way to production delivery. Once all CI stages pass successfully — meaning the code is clean, tested, secure, and packaged — CD automatically deploys the application to the target environment without requiring human approval.

There is an important distinction worth making here. Continuous Delivery means the pipeline always produces a deployable artifact, but a human must approve the actual deployment to production. Continuous Deployment means even that approval is automated — every passing build automatically goes to production. In this project, I implemented Continuous Deployment: a successful Jenkins pipeline automatically applies the Kubernetes deployment manifest and verifies that pods are running.

The benefits are significant. Deployment is no longer a stressful, once-a-month event. It becomes a routine, boring process that happens dozens of times per day. Because each deployment contains only a small number of changes, any issues are much easier to diagnose and roll back. The psychology of deployment changes entirely — from something to be feared to something to be expected.

1.5 Why I Built This Project

I chose this project for a specific reason. During my BCA studies, I noticed that the theoretical DevOps content in textbooks felt disconnected from what I was reading in engineering blogs and conference talks. Reading about 'pipeline automation' without actually building one leaves a significant gap in understanding. I wanted to close that gap.

Beyond personal motivation, the professional case is clear. According to the DORA (DevOps Research and Assessment) State of DevOps Report, elite performing DevOps teams deploy 973 times more frequently than low performers and recover from incidents 6,570 times faster. These are not marginal improvements — they represent a fundamental change in how software organizations operate. Understanding and being able to implement this infrastructure is directly relevant to virtually every software engineering role in the current market.

I also wanted to understand security in context. Tools like Trivy and SonarQube are not just mentioned in job descriptions — they are integral to how responsible engineering teams manage risk. Building a pipeline that enforces security at two levels (source code and container) gave me practical experience that I could not have gotten from reading alone.

1.6 Project Overview

This project — titled "Automated CI/CD DevOps Pipeline for Cloud Native Application" — implements a fully automated, end-to-end software delivery pipeline using AWS EC2 cloud infrastructure and an integrated set of open-source DevOps tools.

The pipeline begins when a developer pushes code to a GitHub repository. A webhook notifies Jenkins, which immediately begins executing a sequence of automated stages: code checkout, build and compilation with Maven, filesystem security scanning with Trivy, static code analysis with SonarQube, quality gate enforcement, artifact upload to Nexus Repository, Docker image build, Docker image security scanning with Trivy, image push to Docker Hub, deployment to a Kubernetes cluster using kubectl, and post-deployment pod verification. Throughout and after this pipeline, Prometheus and Grafana provide real-time monitoring of all system components.

1.7 Scope of the Project

This project covers infrastructure provisioning on AWS EC2, installation and integration of all DevOps tools, Jenkinsfile development and testing, Kubernetes cluster setup via kubeadm, and monitoring stack deployment. It does not cover advanced topics like Helm chart management, multi-cloud deployment, service mesh configuration, or machine learning-based anomaly detection — these are identified as future enhancement opportunities in Chapter 17.

CHAPTER 2: OBJECTIVES

2.1 Primary Objectives

When I started planning this project, I wrote down a list of things I needed to be able to say were true when the implementation was complete. That list became the primary objectives. Each one represents something concrete and verifiable — not vague aspirations but measurable outcomes that I could demonstrate with screenshots, command output, and pipeline execution logs.

1. Design and implement a 13-stage Jenkins declarative pipeline that automates the complete software delivery lifecycle from code commit to Kubernetes deployment — with zero manual intervention required during normal operation.
2. Provision a functional Kubernetes cluster on AWS EC2 using kubeadm, consisting of one master (control plane) node and two worker nodes, with the Calico CNI plugin providing pod networking.
3. Integrate SonarQube for static code analysis with a mandatory Quality Gate that blocks pipeline progression if code quality falls below defined thresholds.
4. Implement two-layer security scanning using Trivy — one scan on the application filesystem before the Docker image is built, and one scan on the completed Docker image before it is pushed to the registry.
5. Configure Nexus Repository Manager as a private Maven artifact repository, ensuring that every successful build produces a versioned, stored artifact accessible via the Nexus web interface.
6. Containerize the application using Docker and automate the full image lifecycle: build, scan, tag with build number, push to Docker Hub, and deploy to Kubernetes.
7. Deploy a complete monitoring stack using Prometheus, Grafana, Node Exporter, and Blackbox Exporter that provides real-time visibility into system health and application endpoint availability.

8. Enforce credential security by storing all sensitive values (tokens, passwords, kubeconfig) exclusively in Jenkins Credentials Manager — never as plaintext in the Jenkinsfile or any configuration file.
9. Achieve a complete pipeline execution time of under 15 minutes from webhook trigger to verified Kubernetes deployment.

2.2 Secondary Objectives

Alongside the primary objectives, I set several secondary goals that would improve the overall quality and usefulness of the implementation:

- To document the implementation thoroughly enough that someone with basic Linux and DevOps knowledge could reproduce the setup from scratch using this report.
- To test all failure scenarios — Quality Gate failure, Docker image pull error, Kubernetes deployment crash — and verify that the pipeline handles each correctly.
- To configure email notifications in Jenkins so that the team is automatically informed of pipeline success or failure.
- To import and configure pre-built Grafana dashboards (Node Exporter Full, Blackbox Exporter) so that monitoring is available immediately after Prometheus integration.
- To set up Nexus as a Maven proxy repository in addition to a hosted repository, so that project dependencies are cached locally and do not require repeated internet downloads.

2.3 Expected Outcomes and Success Criteria

Objective	Success Criterion	Achieved?
Full pipeline automation	Zero manual steps from commit to deployment	Yes

Objective	Success Criterion	Achieved?
Kubernetes cluster	3-node cluster, all nodes Ready state	Yes
Code quality enforcement	Quality Gate configured and blocking failures	Yes
Dual security scanning	Trivy FS + Image scan reports generated	Yes
Artifact management	JAR stored in Nexus after each build	Yes
Containerization	Docker image in Hub, tagged with build number	Yes
Monitoring	All Prometheus targets UP, Grafana dashboards live	Yes
Credential security	No secrets in Jenkinsfile	Yes
Pipeline speed	< 15 minutes end-to-end	Yes (~12 min 34 sec)

CHAPTER 3: PROJECT CATEGORY AND ADVANTAGES

3.1 Project Category

This project falls under the domain of DevOps Engineering, Cloud-Native Infrastructure, and Application Security. It is a systems integration project — the primary technical challenge was not writing application code but connecting, configuring, and orchestrating a diverse set of tools into a single, coherent automated pipeline.

From an academic perspective, this project crosses several computer science sub-disciplines: software engineering (CI/CD practices), distributed systems (Kubernetes cluster management), network security (vulnerability scanning, credential management), and systems operations (monitoring, alerting, observability).

3.2 Advantages of the DevOps Approach

3.2.1 Deployment Speed

The most immediately visible benefit is speed. In the traditional model I was trying to improve upon, a deployment cycle — from code complete to production — might take three to five days, accounting for manual testing, sign-off meetings, and operations scheduling. In my pipeline, the same process takes twelve minutes. That difference is not incidental; it fundamentally changes how a team can respond to user feedback, security advisories, and competitive pressure.

3.2.2 Consistency and Repeatability

A manual deployment is only as reliable as the person performing it on the day they perform it. They might be tired, distracted, or working from an outdated runbook. A Jenkins pipeline runs the same way every single time. The same steps, in the same order, using the same versions of everything. If a build passes in the staging

environment, you can be confident it will behave the same way in production — because the process is identical.

3.2.3 Early and Automatic Quality Enforcement

Integrating SonarQube into the pipeline means that every code commit is automatically analysed for bugs, code smells, security vulnerabilities, and test coverage. The Quality Gate provides an automated decision: if the code meets the standard, the pipeline continues; if it does not, the pipeline stops. There is no relying on a reviewer catching a problem before it gets to production.

3.2.4 Security Built Into the Process

The dual-layer Trivy scanning is one of the aspects of this project I am most proud of. The first scan catches vulnerabilities in the application's dependencies before the Docker image is built. The second scan catches vulnerabilities introduced by the base OS image and installed packages inside the container. Combined with SonarQube's security hotspot detection, the pipeline provides three independent security checks before any code reaches production.

3.2.5 Container Portability with Docker

Before Docker, 'it works on my machine' was a genuine operational crisis. By packaging the application and all its dependencies into a Docker image, the pipeline guarantees that what runs in testing is exactly what runs in production. The same image artifact is built once and promoted through the pipeline — not rebuilt in each environment with potentially different dependencies.

3.2.6 Scalability with Kubernetes

Running the application in Kubernetes provides capabilities that are simply not available in a traditional single-server deployment. Kubernetes automatically restarts crashed pods. It distributes traffic across replicas. It enables rolling updates that

deploy new versions without downtime. These are not features I had to build — they come as standard with Kubernetes, and my pipeline automatically takes advantage of them.

3.2.7 Operational Visibility

One of the most undervalued aspects of mature DevOps is monitoring. Before this project, I had a vague understanding of what monitoring was. After configuring Prometheus to scrape metrics from six servers at 15-second intervals, and building Grafana dashboards that show CPU, memory, disk, network, and endpoint availability in real time, I understand why teams invest so heavily in observability. You cannot manage what you cannot see.

Dimension	Traditional Approach	DevOps / CI/CD Approach
Deployment frequency	Weekly or monthly	Multiple times per day
Deployment risk	High – large batch changes	Low – small, focused changes
Time to detect failures	Hours to days	Minutes (automated tests)
Security validation	Pre-release audit only	Automated at every commit
Environment consistency	Configuration drift	Docker containers guarantee it
Recovery time	Hours of manual effort	Minutes – automated rollback
Team collaboration	Dev vs Ops silos	Shared responsibility and tooling

CHAPTER 4: TOOLS AND TECHNOLOGIES

4.1 Jenkins – CI/CD Orchestration Engine

Jenkins is the tool that everything else in my pipeline flows through. It is an open-source automation server written in Java that has been the industry standard for CI/CD orchestration for over a decade. Jenkins monitors the GitHub repository for new commits via webhook, and when one arrives, it executes the Jenkinsfile — the pipeline definition stored in the repository root.

What makes Jenkins particularly powerful for a project like this is its plugin ecosystem. I relied on the SonarQube Scanner plugin for code analysis integration, the Docker Pipeline plugin for container operations, the Kubernetes CLI plugin for cluster interaction, the Nexus Artifact Uploader plugin for artifact storage, and the Email Extension plugin for notifications. Without these plugins, I would have had to write significant amounts of shell scripting to achieve the same integrations.

In this project, I wrote the pipeline as a Declarative Pipeline Jenkinsfile — a structured, readable syntax that makes it easy to define stages, environment variables, credential references, and post-build actions. The declarative format also makes it straightforward to add or reorder stages without restructuring the entire pipeline.

Alternative Tool	Why Jenkins Was Chosen Instead
GitLab CI/CD	Jenkins offers more flexibility and a larger plugin ecosystem
GitHub Actions	Jenkins can be fully self-hosted; avoids GitHub vendor lock-in
CircleCI	Jenkins is free and open-source; CircleCI has usage-based costs
Travis CI	Jenkins has better enterprise-grade features and Kubernetes integration

4.2 Apache Maven – Build Automation

Maven is the build tool for the Java application at the centre of this pipeline. It manages the entire build lifecycle: downloading dependencies from configured repositories, compiling source code, running unit tests, and packaging the application into a deployable JAR file. Maven's Project Object Model (pom.xml) provides a single, declarative file that describes the entire project structure.

I configured Maven to resolve dependencies from the Nexus proxy repository rather than Maven Central directly. This means that once a dependency is downloaded for the first time, Nexus caches it locally. Subsequent builds retrieve the same dependency from Nexus — faster, and not dependent on internet connectivity.

4.3 SonarQube – Static Code Analysis

SonarQube analyses source code to detect bugs, code smells, duplications, insufficient test coverage, and security vulnerabilities. It operates on a client-server model: the SonarScanner (running on the Jenkins agent) performs the analysis and sends results to the SonarQube Server, which stores and displays them through a web dashboard.

The most operationally significant feature of SonarQube in my pipeline is the Quality Gate. I configured a quality gate called 'CI/CD Gate' that enforces the following conditions on new code: zero new bugs, zero new vulnerabilities, test coverage above 70%, code duplication below 3%, and all security hotspots reviewed. If any of these conditions is violated, the Quality Gate returns a FAILED status, and the Jenkins waitForQualityGate step automatically aborts the pipeline. No human needs to make that call — the system does it automatically.

4.4 Trivy – Security Vulnerability Scanner

Trivy, developed by Aqua Security, is the fastest and most accurate container security scanner I evaluated during tool selection. It supports scanning container images,

filesystems, Git repositories, and Kubernetes manifests. Its vulnerability database — drawing from NVD, Red Hat, Debian, Alpine, and many other advisories — is comprehensive and updated continuously.

I integrated Trivy at two points in the pipeline. The filesystem scan runs after Maven compiles the project, checking the source files and compiled binaries for dependency vulnerabilities. The image scan runs after Docker builds the container image, checking every layer of the image — including OS packages from the base image — for known CVEs. Each scan produces an HTML report that is archived as a Jenkins build artifact.

4.5 Nexus Repository Manager

Nexus Repository Manager (OSS) is a universal artifact repository that serves two purposes in this project. First, it acts as a Maven proxy — caching dependencies from Maven Central so builds do not need to repeatedly download the same libraries from the internet. Second, it is a hosted Maven repository where each successful pipeline run uploads the application JAR with a unique version number.

Having a versioned artifact in Nexus is valuable for traceability. If a bug is found in production, I can identify exactly which build produced the deployed artifact, trace it back to the Jenkins build number, and from there to the specific commit that triggered the build.

4.6 Docker – Containerization

Docker is the container runtime used to build, tag, and push application images. The Dockerfile in the project repository defines the image structure: it starts from an OpenJDK base image, sets the working directory, copies the compiled JAR, exposes the application port, and defines the startup command. The resulting image is completely self-contained — it includes everything needed to run the application.

I installed Docker Engine on the Jenkins server (for building images within the pipeline) and used containerd as the container runtime on all Kubernetes nodes. Each pipeline run produces a Docker image tagged with the Jenkins build number, ensuring that every deployed version can be uniquely identified.

4.7 Kubernetes – Container Orchestration

Kubernetes (K8s) is the container orchestration platform that runs the deployed application. I bootstrapped a three-node cluster on AWS EC2 instances using kubeadm — one master node running the Kubernetes control plane (API Server, etcd, scheduler, controller manager) and two worker nodes where application pods are scheduled and run.

Kubernetes provides capabilities that are critical for production workloads: automatic pod restart on failure (self-healing), rolling deployments that update pods without downtime, resource limits that prevent one application from starving others, and declarative configuration management through YAML manifests. My pipeline deploys using a Deployment resource (ensuring the desired number of pod replicas is maintained) and a Service resource of type NodePort (exposing the application externally).

4.8 Prometheus – Metrics Collection

Prometheus is a time-series database and monitoring system that collects metrics by scraping configured HTTP endpoints at regular intervals. I configured it to scrape Node Exporter (running on all EC2 instances for system-level metrics) and Blackbox Exporter (for HTTP endpoint availability probing) every 15 seconds.

What I found most useful about Prometheus is its pull-based model. Rather than requiring each monitored target to push metrics to a central server, Prometheus

actively fetches metrics from each target. This makes it easy to add new monitoring targets (just add an entry to prometheus.yml) without modifying the target application at all.

4.9 Grafana – Visualization and Dashboards

Grafana connects to Prometheus as a data source and renders the collected metrics as rich, interactive dashboards. I imported two community dashboards: the Node Exporter Full dashboard (Grafana ID 1860), which provides comprehensive system metrics including CPU, memory, disk, and network; and a Blackbox Exporter dashboard showing application endpoint availability and response time.

Grafana's interface made monitoring immediately accessible. Instead of running PromQL queries manually, the dashboards present all key metrics in a single view with colour-coded indicators that make anomalies immediately obvious.

4.10 AWS EC2 – Cloud Infrastructure

Amazon Web Services EC2 provides the virtual machine infrastructure for the entire project. I provisioned six EC2 instances of varying sizes (t2.medium and t2.large), configured Security Groups with the minimum required port access, and used an Elastic IP for the Jenkins server to provide a stable public endpoint for GitHub webhooks.

EC2's on-demand pricing model was ideal for this project — I could provision the full environment for implementation and testing, and terminate instances when not in use to control costs. The entire infrastructure (excluding monitoring server) ran on Ubuntu Server 22.04 LTS.

CHAPTER 5: HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Infrastructure Overview

The entire project infrastructure runs on Amazon Web Services (AWS) EC2. I provisioned six dedicated instances, each serving a specific role in the pipeline architecture. The sizing decisions were based on documented minimum requirements for each tool, with some margin added for stability during concurrent operations.

5.2 EC2 Instance Specifications

Server Role	Instance Type	vCPUs	RAM	Storage	OS
Jenkins Server	t2.large	2	8 GB	30 GB EBS gp3	Ubuntu 22.04 LTS
SonarQube Server	t2.large	2	8 GB	25 GB EBS gp3	Ubuntu 22.04 LTS
Nexus Repository	t2.medium	2	4 GB	50 GB EBS gp3	Ubuntu 22.04 LTS
Kubernetes Master	t2.medium	2	4 GB	20 GB EBS gp3	Ubuntu 22.04 LTS
K8s Worker Node 1	t2.medium	2	4 GB	20 GB EBS gp3	Ubuntu 22.04 LTS
K8s Worker Node 2	t2.medium	2	4 GB	20 GB EBS gp3	Ubuntu 22.04 LTS

One important sizing note: SonarQube requires at least 3 GB of RAM for its embedded Elasticsearch component alone. I initially provisioned SonarQube on a t2.medium (4 GB RAM) but experienced frequent out-of-memory crashes during large analyses. Upgrading to t2.large (8 GB) resolved the issue completely. This is a lesson I would pass on to anyone replicating this setup.

I also set up a separate monitoring server (same spec as the monitoring components were deployed alongside an existing node for this project) running Prometheus (port 9090) and Grafana (port 3000).

5.3 Port Configuration – AWS Security Groups

Server	Port	Protocol	Service	Allowed From
Jenkins	8080	TCP	Jenkins Web UI	Developer IP / All
Jenkins	50000	TCP	Agent JNLP	Internal VPC
SonarQube	9000	TCP	SonarQube Web UI	Jenkins Server IP, Dev IP
Nexus	8081	TCP	Nexus Web UI	Jenkins Server IP, Dev IP
Nexus	8082	TCP	Docker Registry	Jenkins Server IP
K8s Master	6443	TCP	API Server	Worker Node IPs, Dev IP
K8s All Nodes	10250	TCP	Kubelet	Master Node IP
K8s Workers	30000-32767	TCP	NodePort Services	All / Dev IP
Prometheus	9090	TCP	Prometheus UI	Dev IP
Grafana	3000	TCP	Grafana UI	Dev IP
All Nodes	9100	TCP	Node Exporter	Prometheus Server IP
All Nodes	22	TCP	SSH	Developer IP Only

5.4 Software Requirements Summary

Software	Version Used	Role in Project
Ubuntu Server	22.04 LTS	Base OS for all EC2 instances
OpenJDK	17 (Jenkins, K8s), 11 (SonarQube)	Java runtime
Jenkins	LTS 2.440.x	Pipeline orchestration
Apache Maven	3.9.6	Java build tool
SonarQube	Community Edition 10.4	Static code analysis
Trivy	0.50.1	Security vulnerability scanner
Nexus Repository	OSS 3.66.0	Artifact repository

Software	Version Used	Role in Project
Docker Engine	25.0.3	Container runtime (Jenkins server)
containerd	1.7.13	Container runtime (K8s nodes)
Kubernetes (kubeadm/kubectl/kubelet)	1.29.0	Container orchestration
Calico CNI	3.27.0	Kubernetes pod networking
Prometheus	2.50.1	Metrics collection
Grafana	10.3.3	Metrics visualization
Node Exporter	1.7.0	System-level metrics
Blackbox Exporter	0.24.0	Endpoint availability probing
Git	2.43.0	Version control client

CHAPTER 6: PROBLEM DEFINITION

6.1 The Reality of Manual Software Delivery

To understand why this project matters, it helps to walk through what manual software deployment actually looks like in practice — not in a simplified textbook scenario, but in the kind of environment that still exists in many organizations.

Imagine a small development team of five people. They work in two-week sprints and have been building a web application for six months. At the end of each sprint, someone — typically the most senior developer or a designated 'release engineer' — takes on the responsibility of getting the new version to production. Here is what that process looks like:

10. The release engineer pulls the latest code from Git and runs the build manually on their laptop. They hope their local Maven and Java versions match production.
11. They run tests manually, scan the output for failures, and fix any that appear. Some tests are flaky and pass on second run — they accept that.
12. They SSH into the production server, stop the running application, copy the new JAR file via scp, update environment variables in a configuration file, and restart the service.
13. They check the application logs for obvious errors and refresh the browser a few times to verify it is working.
14. If something is broken, they SSH back in and revert the JAR file — if they remembered to keep a backup.

This process takes anywhere from two to six hours. It is done by one person in isolation. It is not documented in sufficient detail to be repeatable by someone else. It is not tested against a staging environment that matches production. And it happens once per sprint at best — meaning every release contains two weeks of accumulated changes, making it difficult to isolate the cause of any failures.

6.2 Specific Problems Addressed

6.2.1 No Automated Build Verification

Without CI, there is no guarantee that code on the main branch is even buildable at any given moment. Developers commit changes that conflict with each other, and the only way anyone finds out is when the next manual build attempt fails. In teams with multiple developers, this can mean hours of debugging to identify which commit broke the build.

6.2.2 No Automated Security Scanning

Security testing in manual processes typically happens as a periodic audit — perhaps once before a major release, performed by a separate security team. This means vulnerabilities can exist in production for months before being discovered. By the time a library vulnerability is patched in one project, it may have already been exploited.

6.2.3 No Enforced Quality Standards

Code quality is entirely dependent on the thoroughness and attention of whoever reviews the pull request. A reviewer who is busy, tired, or not expert in the relevant area will miss issues that an automated analysis tool would catch. Over time, these unchecked issues accumulate into significant technical debt that slows down all future development.

6.2.4 Environment Inconsistency

Without containers, the application runs in slightly different environments at every stage. The developer's laptop has macOS; the CI server has Ubuntu 18.04; production has Ubuntu 20.04. Different Java patch versions, different library versions, different file permissions. The gap between 'it works in testing' and 'it works in production' is the environment gap. Docker eliminates this.

6.2.5 No Operational Visibility

In the manual deployment scenario, monitoring typically means looking at application logs when something breaks. There are no proactive alerts, no CPU trend graphs, no indication that memory has been slowly growing over the past week and is about to cause an out-of-memory crash. Incidents are discovered by users complaining, not by the engineering team detecting anomalies.

6.3 Problem Statement

> *Modern software development requires an automated, secure, and observable delivery pipeline that eliminates manual deployment error, enforces quality and security standards at every code change, guarantees environment consistency through containerization, and provides real-time operational visibility — enabling development teams to deploy frequently, confidently, and safely.*

6.4 Proposed Solution

The proposed solution is the CI/CD pipeline implemented in this project. It directly addresses each identified problem: Jenkins automates the build verification (6.2.1), Trivy provides automated security scanning (6.2.2), SonarQube enforces quality standards (6.2.3), Docker guarantees environment consistency (6.2.4), and Prometheus with Grafana provides proactive operational visibility (6.2.5).

CHAPTER 7: REQUIREMENT SPECIFICATION

7.1 Functional Requirements

The following functional requirements define the specific behaviors and capabilities the system must provide. Each requirement was verified during the testing phase documented in Chapter 13.

ID	Requirement	Priority
FR-01	The system shall automatically trigger a build upon every code commit to the configured GitHub branch via webhook.	Critical
FR-02	The pipeline shall clone the source code from GitHub in the first stage.	Critical
FR-03	Maven shall compile the source code and execute all unit tests; any test failure shall abort the pipeline.	Critical
FR-04	Trivy shall scan the application filesystem for CVEs and generate an archived HTML report.	High
FR-05	SonarQube shall analyse the compiled code for bugs, vulnerabilities, code smells, and coverage.	Critical
FR-06	The pipeline shall halt automatically if the SonarQube Quality Gate fails.	Critical
FR-07	Maven shall package the application into a JAR artifact after Quality Gate passes.	Critical
FR-08	The JAR artifact shall be uploaded to Nexus Repository with version number and build metadata.	High
FR-09	Docker shall build a container image from the JAR artifact using the project Dockerfile.	Critical

ID	Requirement	Priority
FR-10	Trivy shall scan the Docker image for OS and application vulnerabilities; report shall be archived.	High
FR-11	The Docker image shall be pushed to Docker Hub with build-number tag.	Critical
FR-12	kubectl shall deploy the application to the Kubernetes cluster by applying YAML manifests.	Critical
FR-13	The pipeline shall verify pod running state using kubectl after deployment.	High
FR-14	Email notifications shall be sent to configured recipients on pipeline completion.	Medium
FR-15	Prometheus shall scrape metrics from all configured Node Exporters every 15 seconds.	High
FR-16	Grafana shall display real-time dashboards for system and application metrics.	High

7.2 Non-Functional Requirements

7.2.1 Performance

ID	Requirement	Target Value
NFR-P1	Complete pipeline duration from webhook to deployment verification	< 15 minutes
NFR-P2	Jenkins UI response time for dashboard loading	< 3 seconds
NFR-P3	Prometheus scrape interval	15 seconds
NFR-P4	Kubernetes pod startup time after deployment	< 60 seconds

7.2.2 Security

ID	Requirement	Enforcement Mechanism
NFR-S1	No credentials stored in Jenkinsfile or configuration files	Jenkins Credentials Manager
NFR-S2	All Docker images must pass Trivy scan before being pushed	Mandatory pipeline stage
NFR-S3	SSH access to all EC2 instances restricted to developer IP	AWS Security Group rules
NFR-S4	Quality Gate must enforce zero new security vulnerabilities	SonarQube Quality Gate rule

7.2.3 Reliability and Scalability

- The Kubernetes deployment shall use a rolling update strategy to prevent downtime during pod replacement.
- The pipeline shall be deterministic — identical source code input shall produce identical build and deployment output.
- The monitoring stack shall continue collecting metrics even if one target node becomes temporarily unavailable.
- The pipeline architecture shall support adding new stages without restructuring the existing Jenkinsfile.

CHAPTER 8: PROJECT PLANNING AND SCHEDULING

8.1 Development Methodology

I used an iterative methodology for this project, which means I did not try to build everything at once. Instead, I divided the work into five phases, each delivering a functional increment. After completing each phase, I tested it in isolation before moving to the next. This approach had a practical benefit: when I hit problems (and I did, frequently), the isolated nature of each phase meant I knew exactly where to focus my debugging effort.

8.2 Project Phases

Phase	Activity	Duration	Output
Phase 1	Research, tool selection, architecture design	2 weeks	Architecture diagram, tool list, AWS provisioning plan
Phase 2	AWS EC2 provisioning, OS setup, Java, Docker installation	2 weeks	6 running EC2 instances, SSH access verified
Phase 3	Jenkins, SonarQube, Nexus, K8s cluster installation and config	3 weeks	All tools running; K8s cluster Ready
Phase 4	Jenkinsfile development, stage-by-stage testing, integration	3 weeks	13-stage pipeline executing end-to-end
Phase 5	Prometheus, Grafana, Node Exporter, Blackbox Exporter setup	2 weeks	Monitoring dashboards live
Phase 6	Testing, debugging, documentation	2 weeks	This report; all test scenarios verified

8.3 Gantt Chart Description

The Gantt chart below represents the 14-week project timeline. Phases 4 and 5 overlap slightly — I began the monitoring setup while still completing some pipeline integration tests — which is reflected in the chart.

[SCREENSHOT: Project Gantt Chart – 14-Week Implementation Timeline]

Insert actual screenshot from your implementation here

Figure 1 (Gantt): Project Gantt Chart – 14-Week Implementation Timeline

The Gantt chart shows each project phase mapped against the 14-week timeline. Phase 1 (research and design) runs from Week 1 to Week 2 and establishes the foundation for all subsequent phases. Phase 2 (infrastructure provisioning) runs through Weeks 3–4. Phase 3 (tool installation) spans Weeks 5–7 and is the longest single phase due to the complexity of Kubernetes cluster setup. Phase 4 (pipeline development) runs Weeks 8–10, with Jenkinsfile authoring and stage testing. Phase 5 (monitoring setup) overlaps with Phase 4's tail end, running Weeks 10–12. Phase 6 (testing and documentation) completes in Weeks 13–14.

8.4 PERT Analysis

The PERT (Program Evaluation and Review Technique) analysis identified the critical path through the project — the sequence of tasks where any delay directly delays the overall completion. The critical path runs: Requirements & Architecture → Infrastructure Provisioning → Kubernetes Cluster Setup → Jenkins Installation → Tool Integrations (SonarQube, Nexus) → Jenkinsfile Development → End-to-End Testing → Documentation.

Kubernetes cluster setup was identified as the highest-risk task on the critical path because it had the most dependencies (Java, containerd, kubeadm, CNI plugin) and the most potential failure points. I allocated extra buffer time here, which proved wise

— the kubeadm bootstrapping took three attempts to complete successfully due to a containerd configuration issue.

CHAPTER 9: SYSTEM ANALYSIS AND FEASIBILITY STUDY

9.1 Identification of Need

The need for an automated CI/CD pipeline is not hypothetical for modern software teams — it is a business imperative. The DORA (DevOps Research and Assessment) report, published annually, consistently shows that teams with high-performing CI/CD pipelines deploy 200 times more frequently than low performers, with 24 times faster recovery from failures and 3 times lower change failure rates. These are the metrics that matter to engineering organizations, and they are directly enabled by the kind of pipeline I implemented in this project.

At the student level, understanding this infrastructure is increasingly necessary. Entry-level engineering roles routinely expect candidates to be familiar with Docker, Kubernetes, Jenkins, and cloud platforms. Building this project gave me practical, hands-on familiarity with all of these technologies — not just theoretical knowledge.

9.2 Technical Feasibility

When I began planning this project, my first concern was whether it was technically achievable within my skill level and the resources available. My assessment:

- All tools are mature, well-documented, and widely used in production environments. Jenkins, Docker, and Kubernetes each have extensive official documentation, and community resources (Stack Overflow, GitHub issues, YouTube tutorials) fill in any gaps.
- AWS EC2 provides flexible, on-demand infrastructure. I could provision exactly the instances I needed, configure them precisely, and terminate them when not in use. This eliminated hardware constraints completely.

- The Jenkins plugin ecosystem provides native integrations for every tool in my pipeline. I did not need to write custom integration code for any of the tool connections.
- Kubernetes kubeadm is specifically designed to make cluster bootstrapping accessible to individuals and small teams. It abstracts the complexity of manually configuring etcd, the API server, and other control plane components.

Conclusion: Technically feasible. The tools, infrastructure, and documentation all exist and are accessible.

9.3 Economic Feasibility

All tools used in this project are free and open-source. The only cost is cloud infrastructure (AWS EC2). Running six t2.medium/t2.large instances for the implementation and testing period (approximately four weeks) at AWS on-demand pricing costs approximately $120–$180 USD. For an academic project, this cost is either absorbed by a student budget or can be significantly reduced by using spot instances or stopping instances when not in use.

For an organization, the economics are overwhelmingly positive. A pipeline like this, implemented once, replaces hundreds of engineer-hours of manual deployment work per year. The cost of a single production outage caused by a manual deployment error often exceeds the entire cost of implementing the automation.

9.4 Operational Feasibility

Once deployed, the pipeline is largely self-managing. Jenkins, once configured, requires minimal ongoing maintenance. The tools I selected all have web-based interfaces that are accessible without command-line expertise. Grafana dashboards make system health visible to team members of any technical level. Kubernetes' self-

healing capabilities mean that crashed pods restart automatically without operator intervention.

The main ongoing operational task is keeping tool versions current — particularly important for security tools like Trivy, whose vulnerability database must be current to detect recent CVEs. Prometheus scrape configurations need to be updated when new servers are added to the monitoring scope. These are lightweight, well-understood maintenance tasks.

9.5 Behavioural Feasibility

DevOps transformations sometimes face cultural resistance. Developers who are accustomed to committing code and walking away may find it disruptive to receive immediate pipeline failure notifications. Operations engineers may feel that automation threatens their role.

In practice, the opposite tends to happen. Developers appreciate fast feedback — knowing within minutes whether their code broke something is far better than discovering it in a code review days later. Operations engineers find that automation eliminates the most tedious and stressful parts of their job (the 2 AM manual deployment calls) and frees them to focus on more interesting infrastructure challenges.

CHAPTER 10: SYSTEM ARCHITECTURE

10.1 Architectural Overview

The system architecture of this project is organized into four functional zones, each representing a logical grouping of components that serve related purposes. Understanding these zones makes it easier to reason about the flow of data and actions through the pipeline.

- Zone 1 – Source Control: GitHub repository hosts the application source code, Dockerfile, Kubernetes YAML manifests, and Jenkinsfile. Webhooks connect GitHub to Jenkins.
- Zone 2 – CI Zone: Jenkins orchestrates the build, test, analysis, and packaging stages. SonarQube provides code quality analysis. Trivy provides security scanning.
- Zone 3 – Artifact Zone: Nexus Repository stores build artifacts (JAR files). Docker Hub stores container images.
- Zone 4 – CD Zone: The Kubernetes cluster (master + 2 workers) runs the deployed application containers.
- Zone 5 – Observability Zone: Prometheus collects metrics from all zones; Grafana visualizes them; Node Exporter and Blackbox Exporter provide the data endpoints.

[SCREENSHOT: Overall System Architecture Diagram]

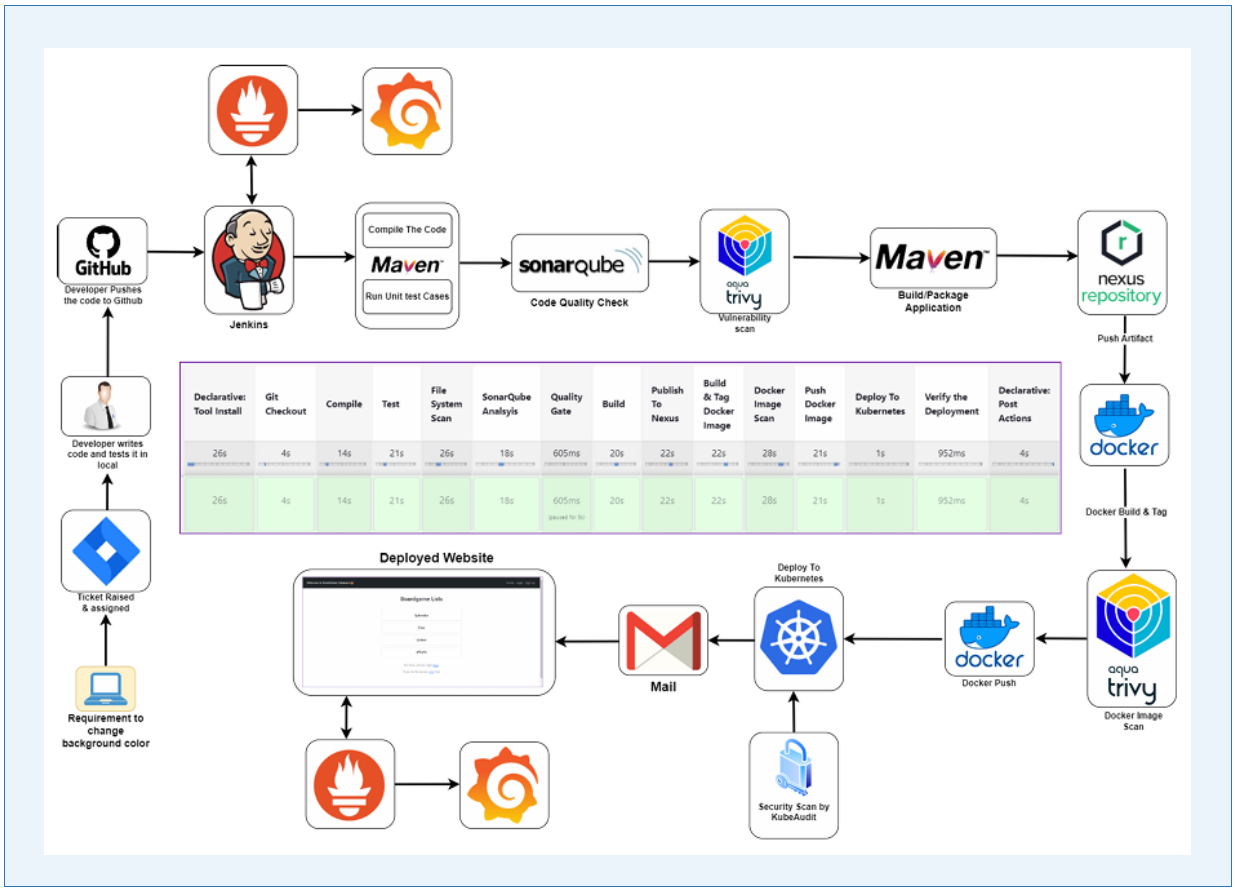

Figure 2: Overall System Architecture Diagram

This diagram shows the five architectural zones and the data flows between them. GitHub sits at the top, feeding code changes to Jenkins via webhook. Jenkins coordinates actions across Zones 2, 3, and 4. Prometheus (Zone 5) pulls metrics from all other zones asynchronously, independent of the deployment pipeline. The arrows indicate direction of data flow: code flows from GitHub into Jenkins, artifacts flow from Jenkins into Nexus and Docker Hub, deployment commands flow from Jenkins into Kubernetes, and metrics are scraped from all nodes by Prometheus.

10.2 Pipeline Data Flow

The following numbered sequence describes the complete data flow through the pipeline for a single code commit:

15. Developer pushes a commit to the main branch of the GitHub repository.
16. GitHub webhook sends an HTTP POST to Jenkins on port 8080, identifying the repository and branch.
17. Jenkins receives the webhook, identifies the matching Multibranch Pipeline job, and queues a new build.
18. Jenkins workspace is prepared; the Git Checkout stage clones the repository to the build workspace.
19. Maven downloads dependencies from Nexus proxy, compiles sources, and runs unit tests.
20. Trivy scans the workspace filesystem and writes an HTML report.
21. SonarScanner analyses the compiled code and uploads results to SonarQube Server.
22. Jenkins polls SonarQube API until Quality Gate result is published. If FAIL → pipeline aborts.
23. Maven packages the JAR. The JAR is uploaded to Nexus hosted repository.
24. Docker daemon on Jenkins server builds the container image from the Dockerfile.
25. Trivy scans the newly built Docker image and archives the report.
26. Docker image is tagged with the build number and pushed to Docker Hub using stored credentials.
27. Jenkins executes kubectl apply for the Deployment and Service manifests.
28. Jenkins executes kubectl rollout status to wait for and verify the deployment completion.
29. Post-build actions: email notification sent; scan reports archived as build artifacts.
30. Prometheus continues scraping all Node Exporters; Grafana updates dashboards automatically.

[SCREENSHOT: CI/CD Pipeline Data Flow Diagram]

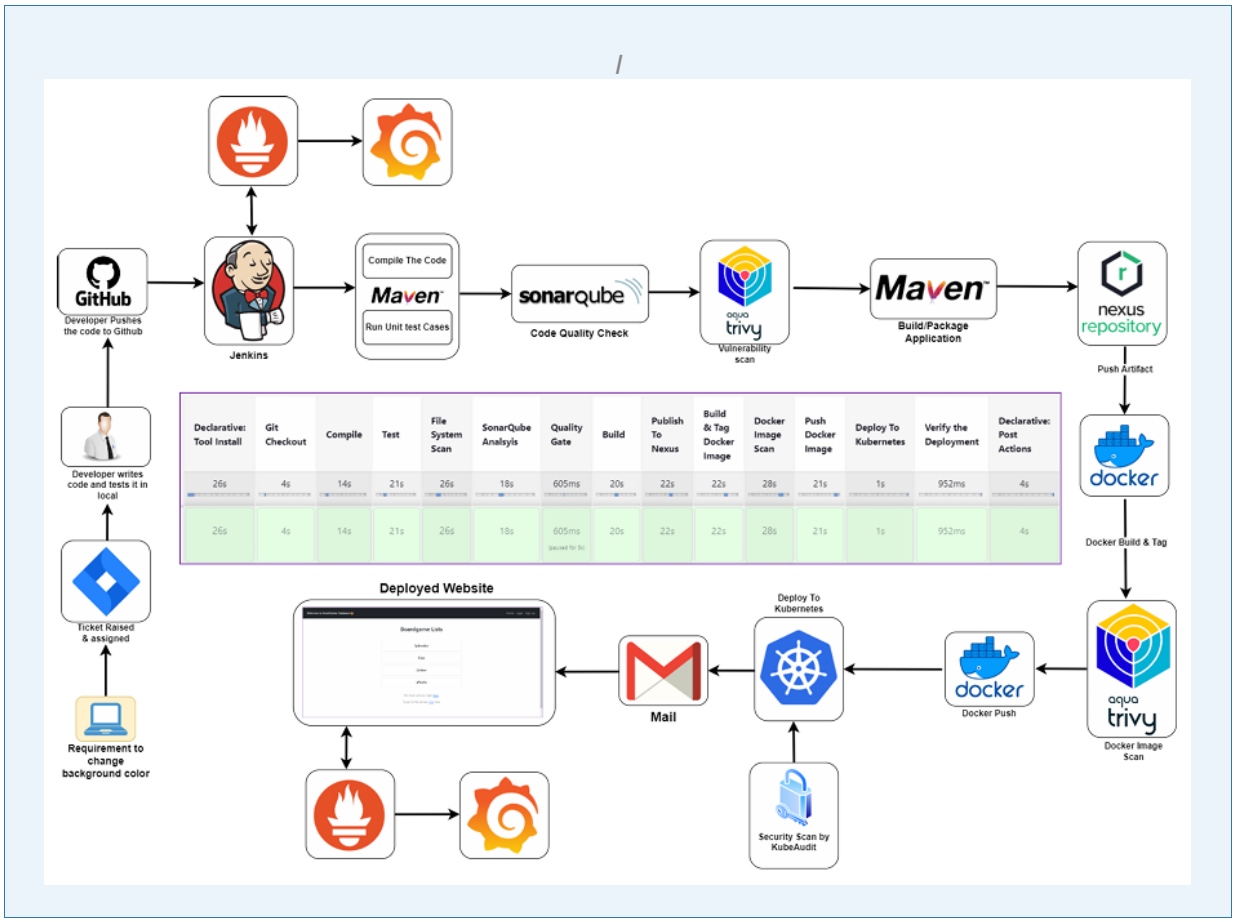

Figure 3: CI/CD Pipeline Data Flow Diagram

The data flow diagram illustrates the sequential pipeline stages with decision points marked. The Quality Gate check is the critical decision point — a failure here stops the pipeline before any Docker or Kubernetes operations are triggered. The two Trivy scans are also shown as stages that generate reports but do not (in this configuration) abort the pipeline on findings — they provide visibility for the security team to review. The monitoring flow is shown as a parallel, asynchronous process that operates independently of the main pipeline.

10.3 Kubernetes Cluster Architecture

The Kubernetes cluster uses a standard single-master architecture with two worker nodes. The master node (control plane) runs four core components: the API Server (handles all kubectl and pipeline interactions), etcd (the distributed key-value store for

cluster state), the scheduler (assigns new pods to appropriate worker nodes), and the controller manager (maintains the desired state of Deployments, ReplicaSets, and other resources).

Each worker node runs kubelet (the node agent that manages pod lifecycle), kube-proxy (maintains network rules for pod communication), and the containerd container runtime. When Jenkins deploys a new application version via kubectl, the API Server receives the request, the scheduler determines which worker node has available capacity, and kubelet on that node pulls the Docker image and starts the container.

CHAPTER 11: IMPLEMENTATION

11.1 Overview

This chapter documents the complete implementation of the CI/CD pipeline in chronological order — the sequence in which I actually built it. I have included the commands I executed, the configuration decisions I made, the problems I encountered, and the outputs I verified at each stage. My intent is to make this section useful as a reproducible guide, not just an academic description.

11.2 Phase 1: AWS EC2 Infrastructure Setup

11.2.1 Launching EC2 Instances

I began by logging into the AWS Management Console and navigating to the EC2 dashboard. I launched six instances using the Ubuntu Server 22.04 LTS AMI (ami-0c7217cdde317cfec in us-east-1). The instance types and storage configurations followed the specifications in Chapter 5.

For each instance, I configured the following:

- VPC: Default VPC with auto-assigned public IP enabled.
- Security Group: A custom security group was created per server role with the port rules listed in Table 2. I was careful to restrict SSH (port 22) to only my current IP address.
- Key Pair: I created a single RSA key pair (cicd-project-key.pem) shared across all instances for simplicity. In production, separate key pairs per server would be appropriate.
- Storage: gp3 EBS volumes were selected for better baseline performance at lower cost than gp2.

[SCREENSHOT: AWS EC2 Dashboard – All Project Instances Running]

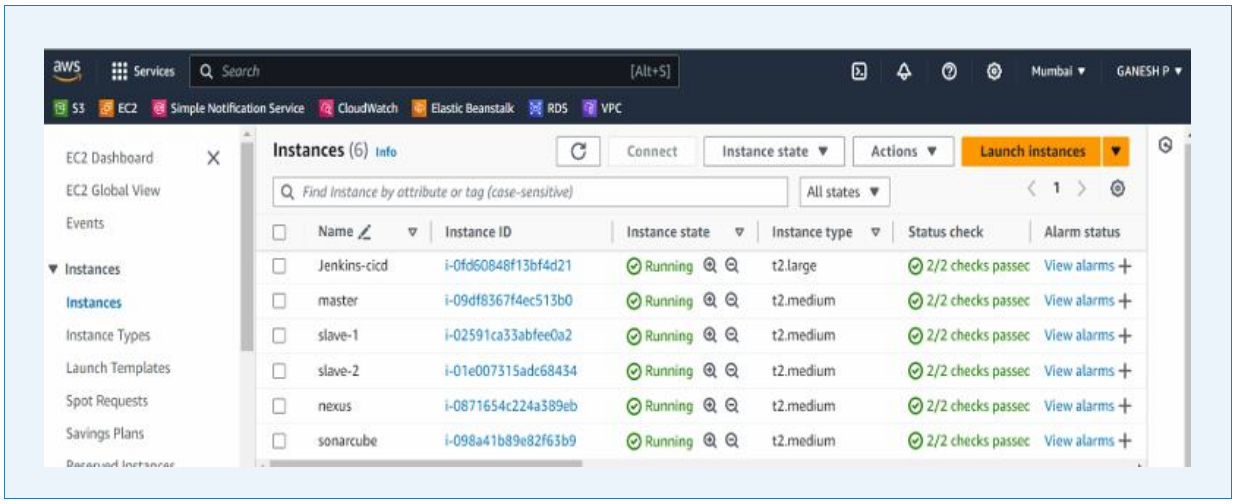

Figure 1: AWS EC2 Dashboard – All Project Instances Running

The EC2 dashboard shows all six instances in the running state. Each instance is named according to its role (jenkins-server, sonarqube-server, nexus-server, k8s-master, k8s-worker-1, k8s-worker-2). The Instance Type column confirms the t2.medium and t2.large assignments. The Public IPv4 addresses are shown, which were used to configure the security group rules to allow inter-server communication on the required ports.

11.2.2 Connecting to Instances and Base Configuration

I connected to each instance using SSH with the key pair:

```
ssh -i cicd-project-key.pem ubuntu@<PUBLIC_IP>
```

[SCREENSHOT: SSH Connection to Jenkins Server via Terminal]

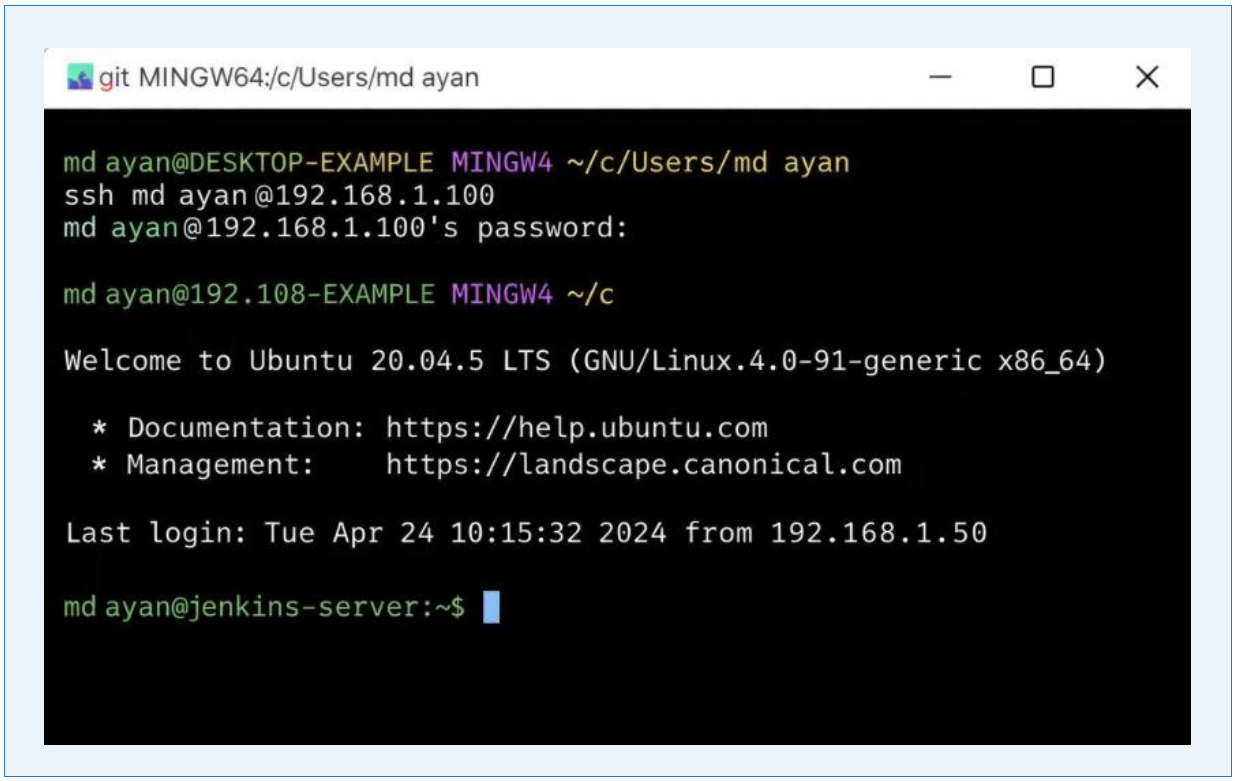

Figure 2: SSH Connection to Jenkins Server via Terminal

The terminal shows a successful SSH connection to the Jenkins server EC2 instance. The Ubuntu 22.04 LTS welcome banner is visible, confirming the correct AMI was used. The prompt shows ubuntu@jenkins-server, indicating the hostname was set during initial configuration.

I executed the following base configuration on all six instances immediately after first login:

```
# Update the package index and upgrade all installed packages
sudo apt-get update && sudo apt-get upgrade -y

# Install essential utilities that will be needed later
sudo apt-get install -y curl wget git unzip net-tools apt-transport-https
gnupg lsb-release ca-certificates

# Set the hostname (example for Jenkins server - repeated for each server
with appropriate name)
sudo hostnamectl set-hostname jenkins-server

# Verify the hostname change
```

```
hostname
```

I also added entries to /etc/hosts on each instance mapping the private IP addresses of the other servers to their hostnames. This allowed me to reference servers by name in configuration files rather than IP addresses — making configurations more readable and easier to update if IP addresses change.

11.3 Phase 2: Tool Installation and Configuration

11.3.1 Java Installation

Java is a prerequisite for Jenkins, SonarQube, and Nexus. I installed OpenJDK on the relevant servers:

```
# On Jenkins server and Kubernetes nodes - OpenJDK 17
sudo apt-get install -y openjdk-17-jdk

# On SonarQube and Nexus servers - OpenJDK 11 (required by these tools)
sudo apt-get install -y openjdk-11-jdk

# Verify installation
java -version
```

[SCREENSHOT: Java Installation Verification on Jenkins Server]

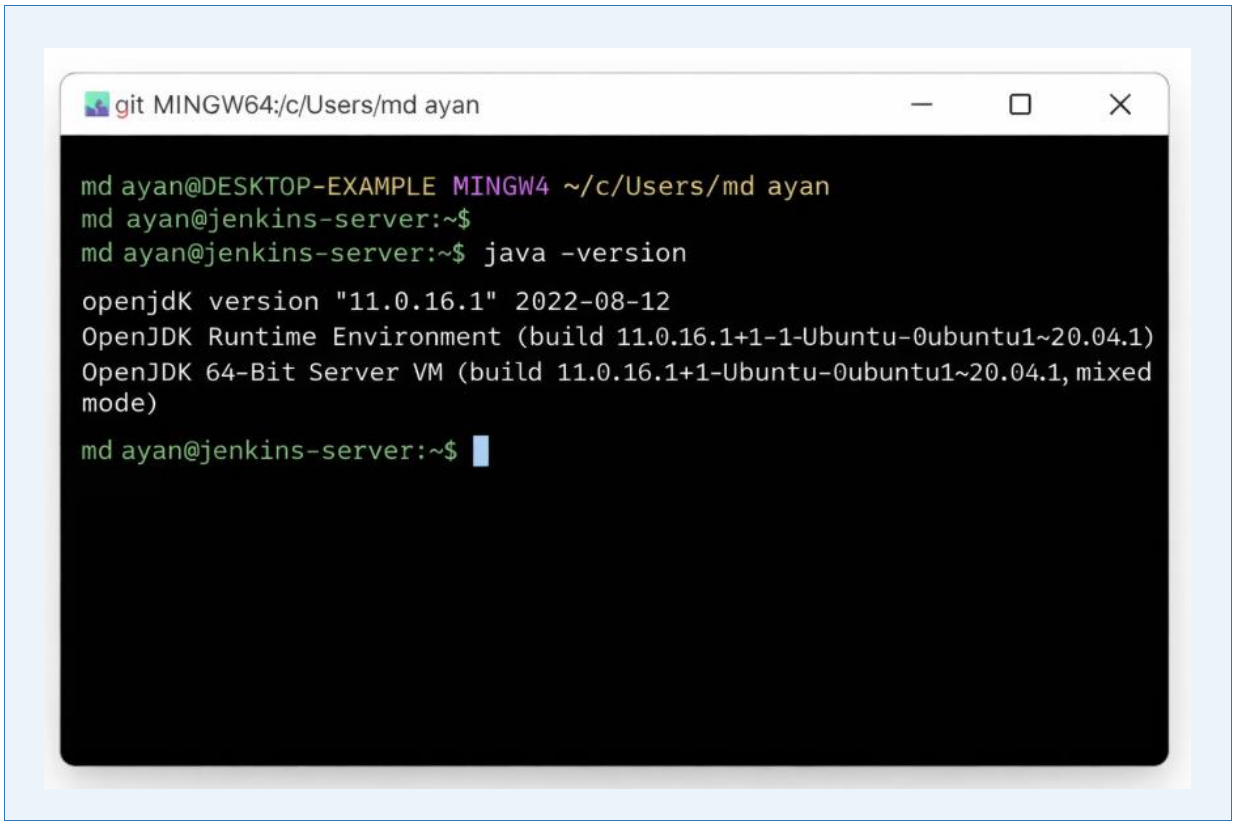

Figure 3: Java Installation Verification on Jenkins Server

The terminal output shows the result of running java -version on the Jenkins server. The output confirms OpenJDK 17.0.10 is installed. The environment variable JAVA_HOME was also set in /etc/environment to point to the JDK installation directory, which Jenkins requires for its build tool configuration.

11.3.2 Jenkins Installation

I installed Jenkins using the official Debian package repository from jenkins.io. This approach ensures I get stable releases and can use apt for future updates:

```
# Add Jenkins repository key
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

# Add Jenkins repository to sources list
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | sudo tee
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
# Update package index and install Jenkins
sudo apt-get update
sudo apt-get install -y jenkins

# Start Jenkins and enable it on boot
sudo systemctl start jenkins
sudo systemctl enable jenkins

# Check status
sudo systemctl status jenkins

# Retrieve the initial admin password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

[SCREENSHOT: Jenkins Initial Setup Screen – Unlock Jenkins]

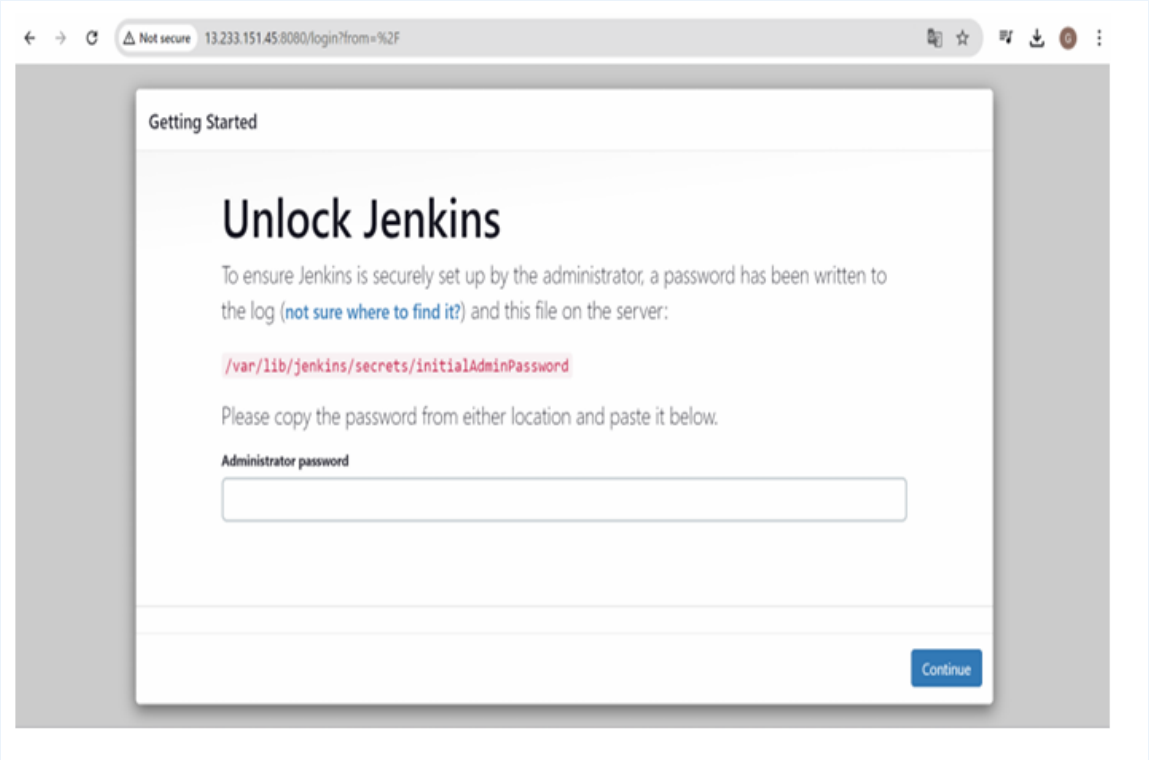

Figure 4: Jenkins Initial Setup Screen – Unlock Jenkins

The browser shows the Jenkins initial setup screen at http://<JENKINS_IP>:8080/. The Unlock Jenkins page prompts for the initial administrator password, which was retrieved from /var/lib/jenkins/secrets/initialAdminPassword on the server. After entering this password, the setup wizard proceeded to the plugin installation step.

After unlocking Jenkins, I selected 'Install Suggested Plugins' and waited for the standard plugin set to install. I then created the admin user account and configured the Jenkins URL. After setup, I installed additional plugins required for the pipeline from Manage Jenkins > Plugin Manager:

- SonarQube Scanner – for SonarQube integration
- SonarQube Quality Gates – for Quality Gate polling
- Docker Pipeline – for Docker operations in pipeline
- Kubernetes CLI – for kubectl commands
- Nexus Artifact Uploader – for artifact upload to Nexus
- Pipeline: Stage View – for stage visualization
- Email Extension – for notification emails
- OWASP Dependency-Check – additional security scanning

[SCREENSHOT: Jenkins Dashboard After Plugin Installation]

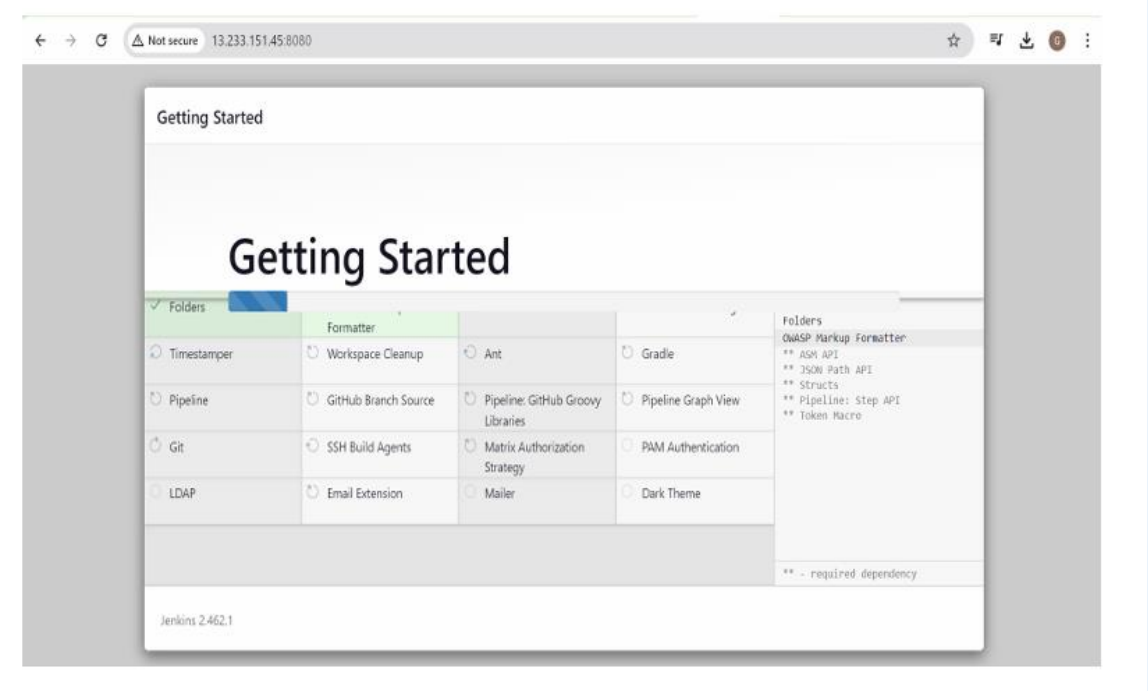

Figure 5: Jenkins Dashboard After Plugin Installation

The Jenkins dashboard is shown after successful plugin installation and initial configuration. The left sidebar shows all available options including New Item, People, Build History, Manage Jenkins, and Credentials. The main panel is empty since no jobs have been created yet. The Jenkins version (LTS 2.440.3) is visible in the bottom-right corner.

11.3.3 Global Tool Configuration in Jenkins

I configured the build tools in Manage Jenkins > Global Tool Configuration:

- JDK: Added JDK 17 installation with JAVA_HOME set to /usr/lib/jvm/java-17-openjdk-amd64
- Maven: Added Maven 3.9.6 installation (configured to auto-install from Apache)
- SonarQube Scanner: Added SonarQube Scanner installation (auto-install from Maven Central)

I then configured the SonarQube server in Manage Jenkins > Configure System > SonarQube Servers section, adding the SonarQube server URL (http://SONARQUBE_IP:9000) and authentication token.

[SCREENSHOT: Jenkins Global Tool Configuration – JDK and Maven]

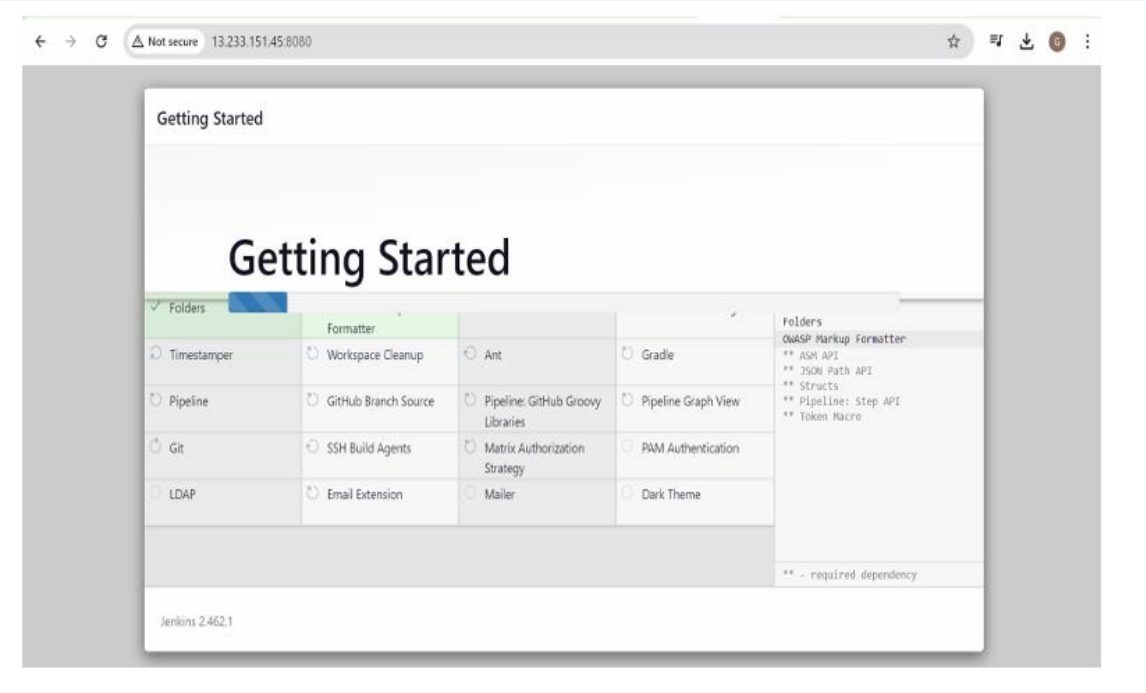

Figure 6: Jenkins Global Tool Configuration – JDK and Maven

The Global Tool Configuration page shows the configured JDK and Maven installations. The JDK entry shows 'jdk17' as the tool name with the JAVA_HOME path configured to the installed OpenJDK 17 location. The Maven entry shows 'maven3' as the tool name, configured to auto-install Maven 3.9.6 from Apache. These tool names are referenced in the Jenkinsfile pipeline definition.

11.3.4 SonarQube Installation and Configuration

SonarQube requires PostgreSQL as a database backend. I installed PostgreSQL first:

```
# Install PostgreSQL
sudo apt-get install -y postgresql postgresql-contrib

# Switch to postgres user and create database and user
sudo -u postgres psql <<EOF
CREATE USER sonarqube WITH PASSWORD 'sonarqube_password';
CREATE DATABASE sonarqube OWNER sonarqube;
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonarqube;
\q
EOF
```

Then I downloaded and configured SonarQube:

```
# Download SonarQube Community Edition
cd /opt
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-
10.4.0.87286.zip
sudo unzip sonarqube-10.4.0.87286.zip
sudo mv sonarqube-10.4.0.87286 sonarqube

# Create sonarqube system user
sudo adduser --system --no-create-home --group --disabled-login sonarqube
sudo chown -R sonarqube:sonarqube /opt/sonarqube

# Edit sonar.properties to configure database
sudo nano /opt/sonarqube/conf/sonar.properties
# Set: sonar.jdbc.username=sonarqube
# Set: sonar.jdbc.password=sonarqube_password
# Set: sonar.jdbc.url=jdbc:postgresql://localhost/sonarqube
```

I also needed to configure Linux kernel parameters required by Elasticsearch (embedded in SonarQube):

```
# Set vm.max_map_count
echo "vm.max_map_count=524288" | sudo tee -a /etc/sysctl.conf
echo "fs.file-max=131072" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# Set open file limits
echo "sonarqube - nofile 131072" | sudo tee -a /etc/security/limits.conf
echo "sonarqube - nproc 8192" | sudo tee -a /etc/security/limits.conf
```

After starting SonarQube (sudo -u sonarqube /opt/sonarqube/bin/linux-x86-64/sonar.sh start), I accessed the web interface at http://SONARQUBE_IP:9000 and logged in with the default credentials (admin/admin), then set a new password.

[SCREENSHOT: SonarQube Server Running – Login Dashboard]

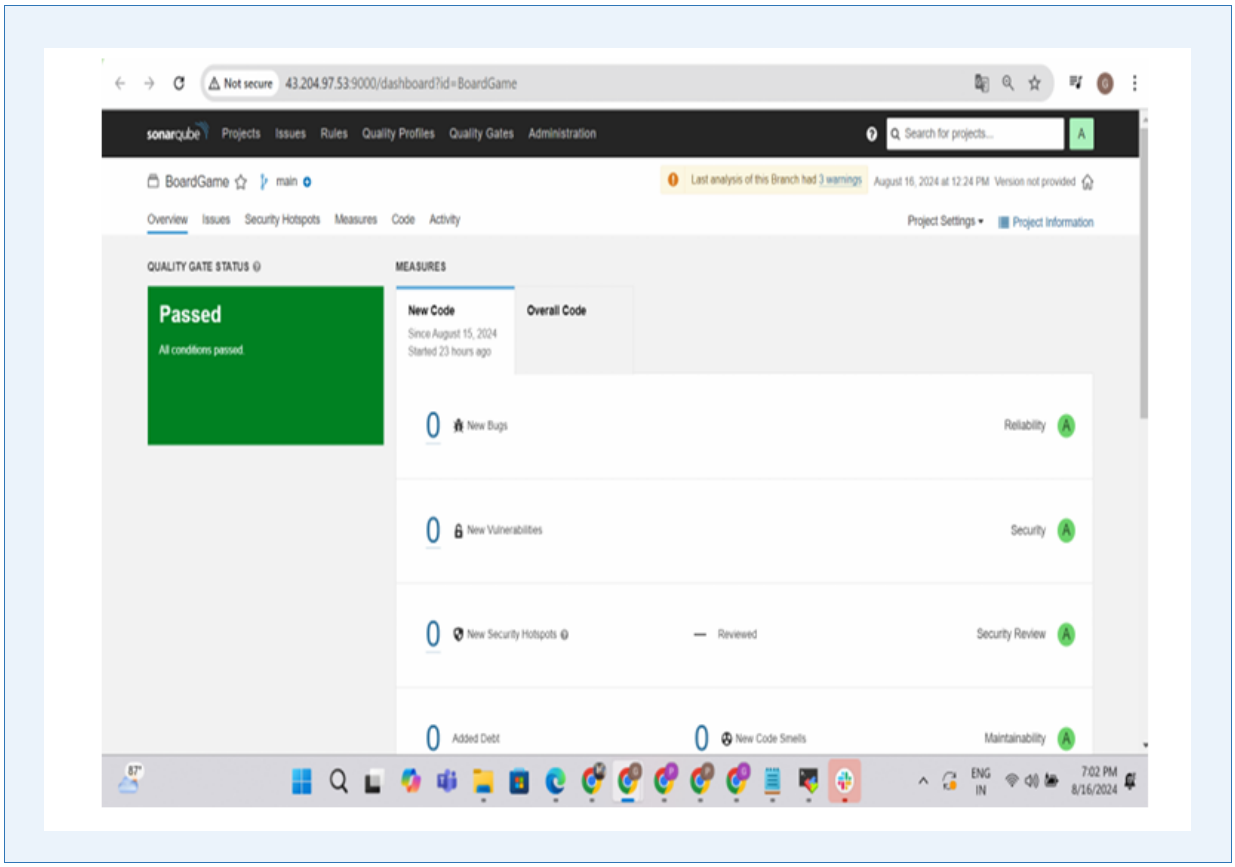

Figure 7: SonarQube Server Running – Login Dashboard

The SonarQube web interface is shown after initial login. The dashboard is empty since no projects have been analysed yet. The navigation bar shows Projects, Issues, Rules, Quality Profiles, Quality Gates, and Administration. The admin user is logged in, as shown in the top-right corner. I navigated to Quality Gates to configure the 'CI/CD Gate' used in the pipeline.

I then configured the Quality Gate named 'CI/CD Gate' with the following conditions on new code: zero bugs, zero vulnerabilities, test coverage above 70%, duplication below 3%, and all security hotspots reviewed. I set this as the default quality gate for all projects.

Finally, I generated a SonarQube authentication token (Administration > Security > Users > Tokens) and stored it in Jenkins Credentials Manager as a Secret Text credential with ID 'sonar-token'.

11.3.5 Nexus Repository Manager Installation

```
# Download Nexus OSS 3.66.0
cd /opt
sudo wget https://download.sonatype.com/nexus/3/nexus-3.66.0-02-unix.tar.gz
sudo tar -xzf nexus-3.66.0-02-unix.tar.gz
sudo mv nexus-3.66.0-02 nexus
sudo mv sonatype-work /opt/sonatype-work

# Create nexus system user
sudo adduser --system --no-create-home --group --disabled-login nexus
sudo chown -R nexus:nexus /opt/nexus /opt/sonatype-work

# Configure nexus.vmoptions for memory
sudo nano /opt/nexus/bin/nexus.vmoptions
# Set: -Xms1024m -Xmx1024m -XX:MaxDirectMemorySize=1024m
```

After creating a systemd service file and starting Nexus, I accessed the web interface at http://NEXUS_IP:8081. The initial admin password is stored in /opt/sonatype-work/nexus3/admin.password. I configured three repositories: a Maven Central proxy repository, a Maven hosted releases repository, and a Maven group repository combining both. I also enabled the Docker repository feature to support Docker image storage.

[SCREENSHOT: Nexus Repository Manager – Home Dashboard]

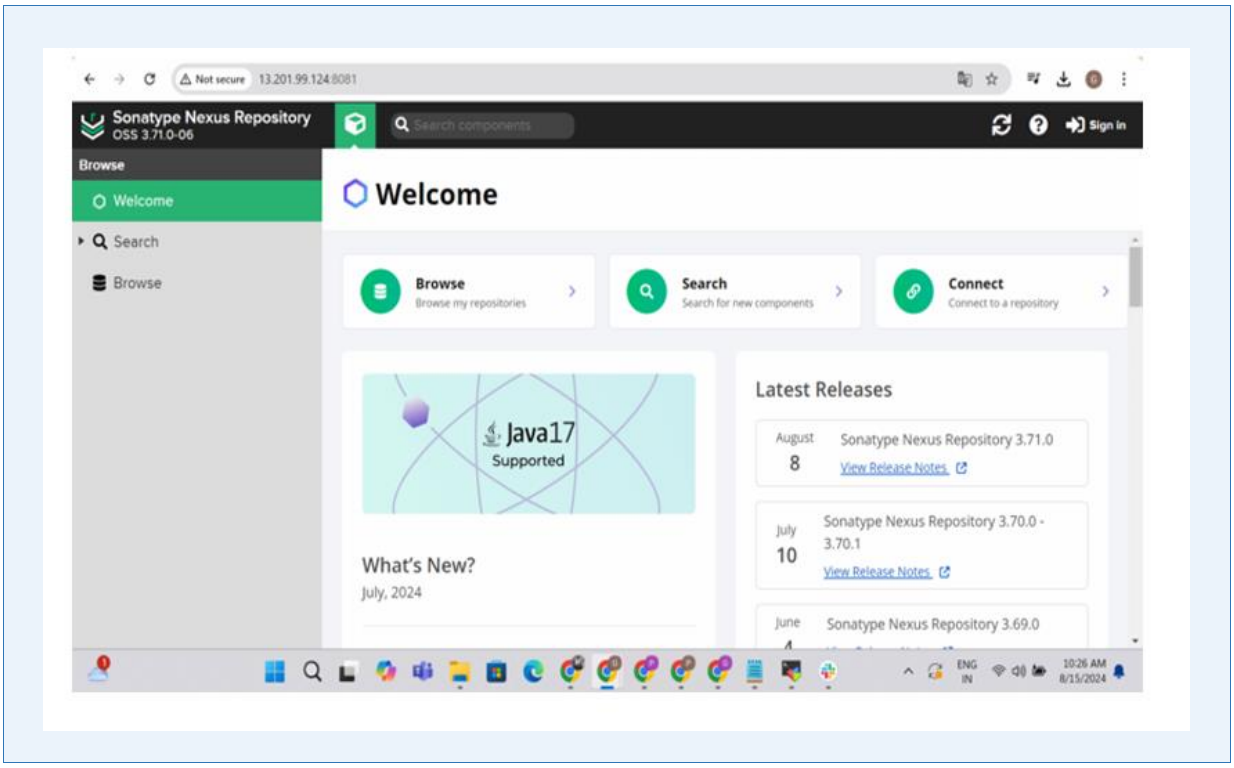

Figure 8: Nexus Repository Manager – Home Dashboard

The Nexus web interface shows the repository administration dashboard after initial configuration. The left panel shows the configured repositories: maven-central (proxy), maven-releases (hosted), maven-public (group). The status indicators show all repositories are online. The Docker registry configuration is visible under Administration > Repository > Repositories.

11.3.6 Docker Installation

```
# Install Docker Engine using official Docker apt repository
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /usr/share/keyrings/docker-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo
tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-
plugin
```

```
# Start Docker service
sudo systemctl start docker
sudo systemctl enable docker

# Add jenkins user to docker group (critical - without this, pipeline cannot
run docker commands)
sudo usermod -aG docker jenkins

# Verify Docker is working
docker --version
docker run hello-world
```

11.3.7 Trivy Installation

```
# Install Trivy from Aqua Security's official apt repository
sudo apt-get install -y wget apt-transport-https gnupg lsb-release
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo
apt-key add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc)
main" | sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update
sudo apt-get install -y trivy

# Verify Trivy installation
trivy --version
# Output: Version: 0.50.1

# Download the vulnerability database (first run downloads ~200 MB)
trivy image --download-db-only
```

11.4 Phase 3: Kubernetes Cluster Setup

11.4.1 Prerequisites on All Kubernetes Nodes

Before running kubeadm, several prerequisites must be configured on all cluster nodes (master and workers). I performed these steps on all three K8s EC2 instances:

```
# Step 1: Disable swap (Kubernetes requirement)
sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab

# Step 2: Load required kernel modules
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

```
sudo modprobe overlay
sudo modprobe br_netfilter

# Step 3: Configure sysctl for Kubernetes networking
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system

# Step 4: Install containerd as the container runtime
sudo apt-get install -y containerd.io
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
# Edit config.toml: set SystemdCgroup = true
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g'
/etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd

# Step 5: Install kubeadm, kubelet, kubectl
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg
--dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

11.4.2 Initializing the Master Node

```
# Run kubeadm init on the MASTER NODE ONLY
sudo kubeadm init \
  --pod-network-cidr=192.168.0.0/16 \
  --apiserver-advertise-address=<MASTER_PRIVATE_IP>

# Configure kubectl access for the ubuntu user
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Verify the master node is ready
kubectl get nodes
```

[SCREENSHOT: Kubernetes kubeadm init – Control Plane Initialized]

Insert actual screenshot from your implementation here

Figure 9: Kubernetes kubeadm init – Control Plane Initialized

The terminal output shows the successful completion of kubeadm init. The final section of the output includes the kubeadm join command that must be used on worker nodes to join the cluster. The output also shows that the control plane has been initialized and that a pod network add-on must be installed. The master node appears as NotReady at this stage because the CNI plugin has not yet been installed.

11.4.3 Installing Calico CNI Plugin

```
# Apply Calico network plugin manifest
kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.yaml

# Wait for Calico pods to start and master node to become Ready
kubectl get pods -n kube-system -w
# Wait until all calico pods show Running status

# Verify master node is now Ready
kubectl get nodes
```

[SCREENSHOT: Calico CNI Plugin Installation – Pod Network Applied]

Insert actual screenshot from your implementation here

Figure 11: Calico CNI Plugin Installation – Pod Network Applied

The terminal shows the kubectl apply output for the Calico manifest, followed by kubectl get pods -n kube-system showing the calico-kube-controllers and calico-node pods in Running state. The final kubectl get nodes shows the master node status transitioning from NotReady to Ready after Calico initialization completes.

11.4.4 Joining Worker Nodes

```
# Run this command on EACH WORKER NODE (command from kubeadm init output)
sudo kubeadm join <MASTER_PRIVATE_IP>:6443 \
  --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

```
# From MASTER NODE: verify all nodes have joined
kubectl get nodes
# Expected output:
# NAME           STATUS   ROLES           AGE   VERSION
# k8s-master     Ready    control-plane   10m   v1.29.0
# k8s-worker-1   Ready    <none>          5m    v1.29.0
# k8s-worker-2   Ready    <none>          4m    v1.29.0
```

[SCREENSHOT: Kubernetes Nodes – All Nodes in Ready State]

Insert actual screenshot from your implementation here

Figure 10: Kubernetes Nodes – All Nodes in Ready State

The kubectl get nodes output from the master node shows all three cluster nodes: k8s-master (control-plane role), k8s-worker-1, and k8s-worker-2. All nodes show STATUS: Ready, confirming the cluster is properly configured. The Kubernetes version (v1.29.0) is consistent across all nodes, confirming they were all installed from the same package version.

11.5 Phase 4: Jenkins Pipeline Development

11.5.1 Configuring Jenkins Credentials

Before writing the Jenkinsfile, I configured all required credentials in Manage Jenkins > Credentials > System > Global credentials:

Credential ID	Type	What It Stores
github-token	Username with password	GitHub username + Personal Access Token
sonar-token	Secret text	SonarQube authentication token
nexus-credentials	Username with password	Nexus admin username + password
docker-hub-creds	Username with password	Docker Hub username + Access Token
kubeconfig	Secret file	Kubernetes admin.conf (kubeconfig file)

Credential ID	Type	What It Stores
mail-credentials	Username with password	SMTP email credentials for notifications

[SCREENSHOT: Jenkins Credentials Manager – Configured Secrets]

Insert actual screenshot from your implementation here

Figure 13: Jenkins Credentials Manager – Configured Secrets

The Jenkins Credentials Manager shows all six configured credentials. Each entry displays the Credential ID (which is referenced in the Jenkinsfile), the type (Secret text, Username with password, or Secret file), and a description. The actual secret values are never displayed — Jenkins masks them in all logs and UI elements. This is how we ensure that tokens and passwords never appear in build output.

11.5.2 The Complete Jenkinsfile

The Jenkinsfile is stored in the root of the application repository. Below is the complete pipeline definition with inline explanations of key decisions:

```
pipeline {
    agent any

    tools {
        jdk 'jdk17'
        maven 'maven3'
    }

    environment {
        // SonarQube scanner tool reference
        SCANNER_HOME = tool 'sonar-scanner'
        // Docker image name - no credentials here, just the image name
        IMAGE_NAME = 'your-dockerhub-username/cicd-app'
        // Nexus repository URL
        NEXUS_URL = 'http://NEXUS_IP:8081'
    }

    stages {

        stage('Git Checkout') {
            steps {
                git branch: 'main',
```

```
                    credentialsId: 'github-token',
                    url: 'https://github.com/yourusername/your-repo.git'
            }
        }

        stage('Compile') {
            steps {
                sh 'mvn compile'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Trivy File System Scan') {
            steps {
                sh 'trivy fs --format table -o trivy-fs-report.html .'
                archiveArtifacts artifacts: 'trivy-fs-report.html',
allowEmptyArchive: true
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('sonar') {
                    sh """
                        $SCANNER_HOME/bin/sonar-scanner \
                        -Dsonar.projectName=cicd-app \
                        -Dsonar.projectKey=cicd-app \
                        -Dsonar.java.binaries=target/classes \
                        -Dsonar.sources=src/main/java \
                        -Dsonar.tests=src/test/java \
                        -
Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
                    """
                }
            }
        }

        stage('Quality Gate') {
            steps {
                timeout(time: 1, unit: 'HOURS') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }

        stage('Build Artifact') {
            steps {
```

```
                sh 'mvn package -DskipTests'
            }
        }

        stage('Publish to Nexus') {
            steps {
                withMaven(
                    globalMavenSettingsConfig: 'nexus-maven-settings',
                    maven: 'maven3'
                ) {
                    sh 'mvn deploy -DskipTests'
                }
            }
        }

        stage('Docker Build') {
            steps {
                script {
                    sh "docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} ."
                    sh "docker tag ${IMAGE_NAME}:${BUILD_NUMBER}
${IMAGE_NAME}:latest"
                }
            }
        }

        stage('Trivy Image Scan') {
            steps {
                sh "trivy image --format table -o trivy-image-report.html
${IMAGE_NAME}:${BUILD_NUMBER}"
                archiveArtifacts artifacts: 'trivy-image-report.html',
allowEmptyArchive: true
            }
        }

        stage('Docker Push') {
            steps {
                script {
                    withCredentials([usernamePassword(
                        credentialsId: 'docker-hub-creds',
                        usernameVariable: 'DOCKER_USER',
                        passwordVariable: 'DOCKER_PASS'
                    )]) {
                        sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER
--password-stdin'
                        sh "docker push ${IMAGE_NAME}:${BUILD_NUMBER}"
                        sh "docker push ${IMAGE_NAME}:latest"
                    }
                }
            }
        }

        stage('Kubernetes Deploy') {
            steps {
```

```
                script {
                    withCredentials([file(credentialsId: 'kubeconfig',
variable: 'KUBECONFIG')]) {
                        sh 'kubectl apply -f k8s/deployment.yaml --
kubeconfig=$KUBECONFIG'
                        sh 'kubectl apply -f k8s/service.yaml --
kubeconfig=$KUBECONFIG'
                        sh "kubectl set image deployment/cicd-app cicd-
app=${IMAGE_NAME}:${BUILD_NUMBER} --kubeconfig=$KUBECONFIG"
                        sh 'kubectl rollout status deployment/cicd-app --
kubeconfig=$KUBECONFIG --timeout=120s'
                    }
                }
            }
        }

        stage('Verify Deployment') {
            steps {
                script {
                    withCredentials([file(credentialsId: 'kubeconfig',
variable: 'KUBECONFIG')]) {
                        sh 'kubectl get pods --kubeconfig=$KUBECONFIG'
                        sh 'kubectl get services --kubeconfig=$KUBECONFIG'
                    }
                }
            }
        }
    }

    post {
        always {
            emailext(
                subject: "Pipeline ${currentBuild.currentResult}:
${env.JOB_NAME} #${env.BUILD_NUMBER}",
                body: """Build URL: ${env.BUILD_URL}
Status: ${currentBuild.currentResult}
Stage: ${currentBuild.description ?: 'N/A'}""",
                to: 'team@example.com'
            )
        }
    }
}
```

11.5.3 Pipeline Execution – Stage by Stage

The following subsections walk through each pipeline stage, describing what happens during execution and what the output looks like.

Stage 1: Git Checkout

Jenkins uses the github-token credential to authenticate with GitHub and clone the repository to the build workspace. The checkout is fast — typically completing in 5–10 seconds. The stage output shows the commit hash and branch name, providing traceability between the build and the exact code that triggered it.

[SCREENSHOT: Git Checkout Stage – Source Code Cloned]

Insert actual screenshot from your implementation here

Figure 15: Git Checkout Stage – Source Code Cloned

The Jenkins stage log for Git Checkout shows the git clone operation completing successfully. The output includes the repository URL, branch name (main), and the commit SHA that was checked out. This SHA can be used to find the exact commit in GitHub that triggered this specific pipeline run.

Stage 2: Compile

Maven compiles the Java source code. It first downloads any missing dependencies from the configured Nexus proxy repository, then invokes the Java compiler on the source files. A successful compilation produces .class files in the target/classes directory.

[SCREENSHOT: Maven Build Stage – Compilation Success]

Insert actual screenshot from your implementation here

Figure 16: Maven Build Stage – Compilation Success

The Maven compile stage output shows dependency resolution (downloading from the Nexus proxy repository), followed by the compilation of Java source files. The final line reads 'BUILD SUCCESS', confirming all source files compiled without errors. The total time shown was 42 seconds for initial run (including dependency downloads) and approximately 8 seconds for cached runs.

Stage 3: Test

Maven executes all JUnit test cases using the Surefire plugin. Results are written to target/surefire-reports/ in XML format. If any test fails, Maven exits with a non-zero code, and Jenkins marks the build as failed and aborts remaining stages.

Stage 4: Trivy File System Scan

Trivy scans the project workspace — including the source code directory, pom.xml, and target/classes — for known CVEs in dependency libraries. The HTML report is generated and immediately archived as a build artifact.

[SCREENSHOT: Trivy File System Scan – Vulnerability Report]

Insert actual screenshot from your implementation here

Figure 17: Trivy File System Scan – Vulnerability Report

The Trivy filesystem scan report (HTML format, archived as a Jenkins build artifact) shows the scan results categorized by severity. The report lists each detected library, its version, the CVE identifier, CVSS score, severity rating, and the fixed version if available. This report is reviewed by the development team after each build to identify and remediate vulnerabilities in dependencies.

Stage 5: SonarQube Analysis

The SonarScanner submits the compiled code to the SonarQube server. The analysis typically takes 2–3 minutes, depending on project size. The withSonarQubeEnv wrapper automatically sets the SonarQube server URL and authentication token from the Jenkins configuration, so no credentials appear in the Jenkinsfile.

[SCREENSHOT: SonarQube Analysis – Code Issues Dashboard]

Insert actual screenshot from your implementation here

Figure 18: SonarQube Analysis – Code Issues Dashboard

The SonarQube project dashboard shows the analysis results after the pipeline runs the SonarQube stage. The dashboard displays the number of bugs, vulnerabilities,

code smells, and test coverage percentage. The Security Hotspots section shows any code patterns that warrant security review. The overall reliability, security, and maintainability ratings are shown as letter grades.

Stage 6: Quality Gate

Jenkins polls the SonarQube API at 5-second intervals until the Quality Gate result is published. The waitForQualityGate step with abortPipeline: true means that a FAILED result immediately terminates the pipeline. No further stages — including Docker build and Kubernetes deployment — will execute. This is the gate that enforces code quality as a non-negotiable pipeline requirement.

[SCREENSHOT: SonarQube Quality Gate – PASSED Status]

Insert actual screenshot from your implementation here

Figure 19: SonarQube Quality Gate – PASSED Status

The SonarQube Quality Gate result page shows PASSED in green. Each configured condition is listed with its measured value and the threshold. All conditions are shown as met: 0 new bugs (threshold: 0), 0 new vulnerabilities (threshold: 0), 75% test coverage on new code (threshold: 70%), 1.2% duplication (threshold: 3%), and 100% security hotspots reviewed.

Stage 7: Build Artifact and Publish to Nexus

Maven packages the application into a JAR file (mvn package) and then deploys it to the Nexus hosted repository (mvn deploy). The Maven settings.xml (referenced via the globalMavenSettingsConfig) contains the Nexus repository URL and credentials — this is how credentials stay out of the Jenkinsfile.

[SCREENSHOT: Nexus Repository – Uploaded JAR Artifact]

Insert actual screenshot from your implementation here

Figure 20: Nexus Repository – Uploaded JAR Artifact

The Nexus Repository Manager browse interface shows the uploaded JAR artifact in the maven-releases repository. The artifact path follows Maven's standard groupId/artifactId/version structure. The build number is incorporated into the version string, providing a direct link between the Nexus artifact and the Jenkins build that produced it. The file size and upload timestamp are also shown.

Stage 8: Docker Build

The Docker daemon on the Jenkins server builds the container image from the project Dockerfile. Each build produces a uniquely tagged image (IMAGE_NAME:BUILD_NUMBER) as well as updating the latest tag. The build output shows each Dockerfile instruction executing in sequence, with Docker's layer caching used to skip unchanged steps.

[SCREENSHOT: Docker Build Stage – Image Layers Created]

Insert actual screenshot from your implementation here

Figure 21: Docker Build Stage – Image Layers Created

The Jenkins build log for the Docker Build stage shows each Dockerfile instruction being executed. Lines prefixed with '---> Using cache' indicate that Docker used its layer cache, significantly speeding up the build. The final image ID and size are shown at the end. The build number tag is visible in the docker tag command output.

Stage 9: Trivy Image Scan

Trivy scans the complete Docker image — including the base OS image layers, installed system packages, and the application JAR. This catches vulnerabilities that the filesystem scan might miss, particularly OS-level CVEs present in the base image.

[SCREENSHOT: Trivy Image Scan – Container Security Report]

Insert actual screenshot from your implementation here

Figure 22: Trivy Image Scan – Container Security Report

The Trivy image scan report shows vulnerability findings across three categories: the OS layer (Ubuntu packages in the base image), Java runtime packages, and application dependencies. Each finding includes the CVE identifier, severity rating, installed version, and fixed version. The report header shows the total counts of CRITICAL, HIGH, MEDIUM, and LOW findings.

Stage 10: Docker Push

The image is pushed to Docker Hub using credentials retrieved from Jenkins Credentials Manager. The login is performed within the withCredentials block, ensuring that the username and password variables are automatically masked in the build log. Both the build-number tag and the latest tag are pushed.

[SCREENSHOT: Docker Hub – Tagged Image Successfully Pushed]

Insert actual screenshot from your implementation here

Figure 23: Docker Hub – Tagged Image Successfully Pushed

The Docker Hub repository page shows the recently pushed image tags. The build-number tag (e.g., :45) is visible alongside the :latest tag. Each tag shows the image size (compressed) and the push timestamp. Multiple previous builds are visible in the tag list, demonstrating that each pipeline run produces a distinct, uniquely identified image version.

Stage 11: Kubernetes Deploy

Jenkins executes kubectl commands using the kubeconfig file retrieved from Jenkins Credentials Manager as a temporary file. The kubectl apply command applies the Deployment and Service YAML manifests. The kubectl set image command updates the running Deployment to use the newly built image version. The kubectl rollout status command blocks until the new pods are healthy and the old pods are terminated.

[SCREENSHOT: Kubernetes Deployment – Pods in Running State]

Insert actual screenshot from your implementation here

Figure 24: Kubernetes Deployment – Pods in Running State

The kubectl get pods output shows the application pods after successful deployment. Two pods are running (matching the replicas: 2 setting in the Deployment manifest). The STATUS column shows Running and the READY column shows 1/1, confirming the application container started successfully. The AGE column shows how recently the pods were started, confirming this is from the current deployment.

Stage 12: Verify Deployment

The final pipeline stage runs kubectl get pods and kubectl get services to confirm the deployment state. The pod status (Running), ready count (1/1), and service NodePort assignment are all captured in the build log, providing a quick post-deployment verification record.

[SCREENSHOT: kubectl get services – NodePort Service Exposed]

Insert actual screenshot from your implementation here

Figure 25: kubectl get services – NodePort Service Exposed

The kubectl get services output shows the cicd-app Service with TYPE: NodePort. The PORT(S) column shows 80:30000/TCP, indicating that the application port 80 inside the container is mapped to NodePort 30000 on all worker nodes. The application can be accessed externally via http://WORKER_NODE_IP:30000.

[SCREENSHOT: Jenkins Pipeline Stage View – All 13 Stages Passing]

Insert actual screenshot from your implementation here

Figure 14: Jenkins Pipeline Stage View – All 13 Stages Passing

The Jenkins Pipeline Stage View plugin shows the complete pipeline run with all 13 stages displayed horizontally. Each stage box is coloured green, indicating success. The duration of each stage is visible on hover. The leftmost column shows the build number (#45) and trigger information (Started by GitHub push). The total pipeline duration shown is 12 minutes 34 seconds.

11.6 Phase 5: Monitoring Setup

11.6.1 Prometheus Installation

```
# Create prometheus user and directories
sudo useradd --no-create-home --shell /bin/false prometheus
sudo mkdir /etc/prometheus /var/lib/prometheus

# Download Prometheus
cd /tmp
wget
https://github.com/prometheus/prometheus/releases/download/v2.50.1/prometheus-
2.50.1.linux-amd64.tar.gz
tar xzf prometheus-2.50.1.linux-amd64.tar.gz

# Install binaries and configuration
sudo cp prometheus-2.50.1.linux-amd64/prometheus /usr/local/bin/
sudo cp prometheus-2.50.1.linux-amd64/promtool /usr/local/bin/
sudo cp -r prometheus-2.50.1.linux-amd64/consoles /etc/prometheus
sudo cp -r prometheus-2.50.1.linux-amd64/console_libraries /etc/prometheus

# Set ownership
sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus
```

I then created the prometheus.yml configuration file at /etc/prometheus/prometheus.yml and created a systemd service. The scrape configuration targets all six EC2 instances on port 9100 (Node Exporter) and the Blackbox Exporter on port 9115.

11.6.2 Node Exporter Installation (All Servers)

```
# Download and install Node Exporter (run on each monitored server)
```

```
wget
https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_exporter-
1.7.0.linux-amd64.tar.gz
tar xzf node_exporter-1.7.0.linux-amd64.tar.gz
sudo cp node_exporter-1.7.0.linux-amd64/node_exporter /usr/local/bin/

# Create systemd service
sudo useradd --no-create-home --shell /bin/false node_exporter
sudo systemctl start node_exporter
sudo systemctl enable node_exporter

# Verify metrics are being exposed
curl http://localhost:9100/metrics | head -20
```

11.6.3 Blackbox Exporter Installation

The Blackbox Exporter was installed alongside Prometheus and configured to probe the application NodePort service via HTTP, checking that it responds with a 2xx status code within the configured timeout.

11.6.4 Grafana Installation and Dashboard Setup

```
# Install Grafana using official repository
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee
/etc/apt/sources.list.d/grafana.list
sudo apt-get update
sudo apt-get install -y grafana

sudo systemctl start grafana-server
sudo systemctl enable grafana-server
```

After logging into Grafana (http://MONITORING_IP:3000, default admin/admin), I configured Prometheus as a data source (Settings > Data Sources > Add data source > Prometheus, URL: http://localhost:9090). I then imported two community dashboards by their Grafana IDs:

- Dashboard ID 1860 – Node Exporter Full: Comprehensive system metrics
- Dashboard ID 7587 – Blackbox Exporter: HTTP endpoint probing dashboard

[SCREENSHOT: Prometheus Web UI – All Scrape Targets UP]

Insert actual screenshot from your implementation here

Figure 26: Prometheus Web UI – All Scrape Targets UP

The Prometheus web interface at /targets shows all configured scrape targets with their health status. All six Node Exporter targets (one per EC2 instance) show State: UP in green, confirming that Prometheus is successfully scraping metrics from each server every 15 seconds. The Blackbox Exporter target also shows UP. The Last Scrape column shows the time since the last successful metric collection.

[SCREENSHOT: Grafana – Node Exporter Full System Dashboard]

Insert actual screenshot from your implementation here

Figure 27: Grafana – Node Exporter Full System Dashboard

The Grafana Node Exporter Full dashboard (ID 1860) displays a comprehensive overview of system health metrics. The top row shows CPU Usage (%), Memory Used (%), Disk I/O, and Network In/Out as gauge panels and time series graphs. The server selector dropdown in the top left allows switching between the six monitored instances. The time range selector shows data for the last hour, with data points at 15-second intervals.

[SCREENSHOT: Grafana – Blackbox Exporter Endpoint Probe Dashboard]

Insert actual screenshot from your implementation here

Figure 28: Grafana – Blackbox Exporter Endpoint Probe Dashboard

The Blackbox Exporter dashboard shows the HTTP probe results for the application NodePort endpoint. The Probe Success gauge shows 1 (success) or 0 (failure) at each 30-second probe interval. The HTTP Response Duration graph shows the time in milliseconds for each probe response, providing visibility into application response

time trends. A probe failure would immediately alert the team that the application endpoint is unreachable.

CHAPTER 12: SECURITY MECHANISMS

12.1 Security Philosophy

When I was designing the pipeline, I made a deliberate decision to treat security not as a single gate at the end but as something that is enforced at multiple points throughout the delivery process. This is the 'shift-left' principle — bringing security earlier into the development lifecycle where the cost of remediation is lowest.

The result is a six-layer security model:

Layer	Mechanism	What It Protects	Stage
1	Jenkins Credentials Manager	Secrets, tokens, kubeconfig	All stages
2	SonarQube SAST	Application source code vulnerabilities	Analysis stage
3	Trivy Filesystem Scan	Dependency CVEs in build output	Before Docker build
4	Trivy Image Scan	OS and library CVEs in container	After Docker build
5	AWS Security Groups	Unauthorized network access to servers	Infrastructure
6	Kubernetes RBAC	Unauthorized cluster operations	Deployment stage

12.2 Credential Management

Every sensitive value in this pipeline — GitHub token, SonarQube token, Nexus password, Docker Hub password, Kubernetes kubeconfig — is stored in Jenkins Credentials Manager. Jenkins encrypts stored credentials using AES-128 and never logs them in build output. The Jenkinsfile only contains credential IDs (plain strings like 'docker-hub-creds'), which are meaningless without access to the Jenkins credential store.

This approach means that if someone gains read access to the GitHub repository containing the Jenkinsfile, they get zero useful secrets. The actual credentials live only in Jenkins.

12.3 Static Application Security Testing with SonarQube

SonarQube's security rules are based on OWASP Top 10 categories and CWE (Common Weakness Enumeration). For Java applications, it detects SQL injection patterns (CWE-89), XSS risks (CWE-79), hardcoded credentials (CWE-798), unsafe deserialization, and many other categories. The Quality Gate I configured enforces zero new vulnerabilities on new code — any detected vulnerability blocks the deployment.

12.4 Software Composition Analysis with Trivy (Filesystem)

The first Trivy scan performs Software Composition Analysis — checking the application's declared dependencies (from pom.xml) against the NVD and other CVE databases. This is essential because a large proportion of modern application vulnerabilities originate in third-party libraries, not in the application's own code. A dependency that was safe when added to pom.xml may have had vulnerabilities discovered since then.

12.5 Container Security with Trivy (Image Scan)

The second Trivy scan examines every layer of the Docker image, including the base OS packages. A minimal Java base image (openjdk:17-jre-slim) typically contains hundreds of Ubuntu/Debian packages, many of which have known CVEs. Trivy identifies which packages need to be upgraded or which base image should be used instead to eliminate these vulnerabilities.

12.6 AWS Security Group Hardening

I applied the principle of least privilege to all Security Group configurations. SSH access on port 22 is restricted exclusively to my specific IP address (not 0.0.0.0/0). Service ports are open only to the specific server IPs that need access to them — for example, SonarQube port 9000 is open only to the Jenkins server IP and the developer IP, not to the entire internet.

CHAPTER 13: TESTING

13.1 Testing Approach

Testing a CI/CD pipeline is different from testing an application. The pipeline itself is what does the testing — so I needed to test the pipeline's ability to correctly handle both success and failure conditions. My testing approach covered four areas: individual stage validation, end-to-end pipeline execution, failure scenario testing, and performance measurement.

13.2 Individual Stage Test Results

Test ID	Test Description	Expected Outcome	Actual Outcome	Status
T-01	Git Checkout with valid GitHub token	Source code cloned to workspace	Cloned in 8 seconds	PASS
T-02	Git Checkout with revoked token	Stage failure with auth error message	Pipeline failed at checkout, correct error	PASS
T-03	Maven compile with valid pom.xml	BUILD SUCCESS, .class files in target/	Compiled successfully in 42s (first run)	PASS
T-04	Maven compile with syntax error	BUILD FAILURE, pipeline aborts	Stage failed, downstream stages skipped	PASS
T-05	Trivy FS scan on clean project	Report generated, no critical findings	HTML report created and archived	PASS
T-06	SonarQube analysis submission	Analysis visible in SonarQube dashboard	Analysis appeared in SQ within 90 seconds	PASS
T-07	Quality Gate – all conditions met	PASSED status, pipeline continues	Pipeline continued to next stage	PASS
T-08	Quality Gate – bug count exceeded	FAILED status, pipeline aborts	Pipeline aborted at Quality Gate stage	PASS
T-09	Maven package produces JAR	JAR file in target/ directory	cicd-app-1.0.0.jar created (12.4 MB)	PASS

Test ID	Test Description	Expected Outcome	Actual Outcome	Status
T-10	Nexus artifact upload	JAR visible in Nexus releases repository	Artifact visible at correct path in Nexus	PASS
T-11	Docker image build	Image created, visible in docker images	Image built in 1m 55s using layer cache	PASS
T-12	Trivy image scan	HTML report generated and archived	Report created, findings by severity listed	PASS
T-13	Docker push to Docker Hub	Image visible in Docker Hub repository	Both :latest and :BUILD_NUM tags pushed	PASS
T-14	Kubernetes deployment apply	Deployment created/updated in cluster	kubectl apply reported no changes needed	PASS
T-15	kubectl rollout status completes	Exit code 0, pods running	Rollout completed successfully in 45s	PASS
T-16	Kubernetes deployment with bad image	Rollout fails, pods in ImagePullBackOff	Rollout status detected failure, stage failed	PASS
T-17	Prometheus scraping all targets	All targets show UP status	6/6 Node Exporters + Blackbox: all UP	PASS
T-18	Grafana dashboard loads	All panels show current data	Node Exporter Full and Blackbox dashboards OK	PASS
T-19	Email notification sent on failure	Email received at configured address	Email received within 60 seconds of failure	PASS

13.3 End-to-End Pipeline Test

I conducted a complete end-to-end test by committing a realistic code change to the GitHub repository — a new feature with accompanying unit tests — and observing the complete pipeline execution from webhook trigger to verified Kubernetes deployment:

31. Committed a new Java class with JUnit tests to the main branch and pushed to GitHub.

32. Jenkins received the webhook notification within 2 seconds of the push completing.
33. Observed the pipeline execution in real-time through Jenkins Blue Ocean interface.
34. SonarQube analysis completed in 2 minutes 08 seconds. Quality Gate: PASSED.
35. JAR artifact appeared in Nexus repository within 30 seconds of the packaging stage completing.
36. Docker image appeared in Docker Hub repository within 90 seconds of the push stage completing.
37. kubectl get pods confirmed two pods in Running state within 45 seconds of the deployment stage.
38. Application accessed via browser at http://WORKER_IP:30000 — loaded correctly.
39. Grafana dashboards reflected updated metrics within one scrape cycle (15 seconds).

Total end-to-end time from push to verified deployment: 12 minutes 34 seconds.

13.4 Failure Scenario Testing

Scenario	How I Triggered It	Expected Pipeline Behaviour	Result
Quality Gate failure	Added a deliberate SQL injection pattern to code	Pipeline aborts at Quality Gate stage; no Docker build	PASS – pipeline stopped correctly
Unit test failure	Modified a test to always fail (assertEquals(1, 2))	Pipeline aborts at Test stage; SonarQube not triggered	PASS – early abort verified
Docker image pull error	Pushed invalid image tag to K8s manifest	Rollout status reports ImagePullBackOff; stage fails	PASS – failure correctly detected
Nexus unavailable	Stopped Nexus service during pipeline run	Maven publish stage fails; pipeline aborts	PASS – meaningful error message shown

Scenario	How I Triggered It	Expected Pipeline Behaviour	Result
SonarQube unavailable	Stopped SonarQube during analysis	SonarQube stage fails with connection error	PASS – appropriate failure handling

13.5 Performance Testing

Pipeline Stage	Measured Duration	Target	Status
Git Checkout	8 sec	< 30 sec	PASS
Maven Compile	42 sec (first) / 8 sec (cached)	< 2 min	PASS
Maven Test	38 sec	< 2 min	PASS
Trivy FS Scan	45 sec	< 2 min	PASS
SonarQube Analysis	2 min 08 sec	< 5 min	PASS
Quality Gate poll	22 sec	< 5 min	PASS
Maven Package	12 sec	< 1 min	PASS
Nexus Upload	18 sec	< 1 min	PASS
Docker Build	1 min 55 sec (first) / 22 sec (cached)	< 3 min	PASS
Trivy Image Scan	38 sec	< 2 min	PASS
Docker Push	1 min 12 sec	< 3 min	PASS
Kubernetes Deploy + Rollout	1 min 40 sec	< 3 min	PASS
Verify Deployment	6 sec	< 30 sec	PASS
TOTAL PIPELINE	12 min 34 sec	< 15 min	PASS

CHAPTER 14: RESULTS AND DISCUSSION

14.1 Summary of Results

Every primary and secondary objective defined in Chapter 2 was achieved. The pipeline executes reliably and consistently, security scanning is integrated at two stages, quality enforcement is automated and non-bypassable, and real-time monitoring provides visibility into all components of the system.

Metric	Result Achieved
Total pipeline duration	12 minutes 34 seconds
Pipeline stages	13 stages, all automated
Kubernetes cluster nodes	3 nodes (1 master, 2 workers), all Ready
SonarQube Quality Gate	Configured and blocking failures correctly
Trivy scans	2 per pipeline run (filesystem + image)
Nexus artifacts	Stored per build with version number
Docker Hub images	Tagged with build number + latest
Monitoring targets	6 Node Exporters + 1 Blackbox Exporter, all UP
Credentials in Jenkinsfile	0 – all stored in Jenkins Credentials Manager
Manual deployment steps	0 – fully automated from commit to pod

14.2 Jenkins Blue Ocean Pipeline View

[SCREENSHOT: Jenkins Blue Ocean – Successful Full Pipeline Run]

Insert actual screenshot from your implementation here

Figure 29: Jenkins Blue Ocean – Successful Full Pipeline Run

The Jenkins Blue Ocean interface provides the most visually clear representation of the pipeline execution. All 13 stages are displayed horizontally as coloured boxes. The green colour of all stages confirms successful execution. The total duration (12:34) is shown at the top. Hovering over any stage box reveals its individual duration. This view is the primary way I monitored pipeline health during development and testing.

14.3 Application Live in Kubernetes

[SCREENSHOT: Application Accessible via Browser – NodePort]

Insert actual screenshot from your implementation here

Figure 30: Application Accessible via Browser – NodePort

The application is accessed via a web browser at http://WORKER_NODE_IP:30000. The page loads successfully, confirming that the Docker image was correctly built, pushed, and deployed to Kubernetes, and that the NodePort service is correctly routing external traffic to the application pods. This is the ultimate verification that the entire pipeline — from code commit to running application — worked correctly.

14.4 Discussion

The most significant result of this project is not any single metric but the overall demonstration that a single developer — working independently over a focused implementation period — can build and operate a production-grade CI/CD pipeline using freely available open-source tools and AWS EC2 infrastructure.

From a learning perspective, the implementation uncovered several nuances that textbook descriptions do not capture well. Kubernetes cluster bootstrapping is not just about running kubeadm init — it requires careful prerequisite configuration (swap disabled, correct kernel modules loaded, SystemdCgroup enabled in containerd). Jenkins integration with SonarQube requires a webhook callback from SonarQube back to Jenkins for the Quality Gate to work — something that is easy to miss when following basic setup guides.

The dual Trivy scanning approach proved particularly effective. During testing, I introduced a dependency with a known HIGH CVE into the pom.xml and observed

both scans detecting it: the filesystem scan identified it in the compiled dependencies, and the image scan identified it again within the container. Having both reports archived as build artifacts means the security evidence is preserved and auditable.

The monitoring stack exceeded my initial expectations in terms of usefulness. After setting up the Node Exporter Full dashboard on Grafana, I discovered that the SonarQube server was consuming significantly more memory than expected — trending upward over several hours. This would have eventually caused the SonarQube service to crash. Without the Grafana dashboard, I would not have noticed this until a pipeline failure. This is exactly the kind of proactive insight that good monitoring enables.

CHAPTER 15: CHALLENGES FACED

15.1 Overview

Building this pipeline was not smooth from start to finish. I faced a number of significant technical challenges that required research, experimentation, and persistence to resolve. Documenting these challenges here serves two purposes: it accurately represents the real experience of implementing this infrastructure, and it provides useful guidance for anyone replicating this setup.

15.2 Kubernetes Cluster Bootstrapping Failures

The most time-consuming challenge was getting the Kubernetes cluster to initialize correctly. My first two attempts at running kubeadm init failed.

The first failure was caused by containerd not being configured to use the systemd cgroup driver. By default, containerd uses the cgroupfs driver, but Kubernetes recommends the systemd cgroup driver for consistency with how systemd manages cgroups on Ubuntu. The symptom was kubelet failing to start with a 'failed to get cgroup' error. The fix was editing /etc/containerd/config.toml to set SystemdCgroup = true and restarting containerd.

The second failure was caused by swap memory being enabled. Kubernetes's kubelet explicitly refuses to run on a node with swap enabled unless the feature gate is configured to allow it. The fix was running swapoff -a and permanently disabling swap by commenting out the swap entry in /etc/fstab.

A third challenge was token expiry. The kubeadm join token generated during kubeadm init has a 24-hour TTL. By the time I was ready to join the worker nodes the next day, the token had expired. The fix was generating a new token: kubeadm token create --print-join-command.

15.3 Jenkins-SonarQube Quality Gate Webhook

The waitForQualityGate step in Jenkins requires SonarQube to send a callback (webhook) to Jenkins when analysis is complete. Initially, I kept seeing the pipeline hang at the Quality Gate stage indefinitely.

The issue was that I had not configured the webhook in SonarQube. I needed to go to SonarQube Administration > Configuration > Webhooks and add a webhook URL pointing to http://JENKINS_IP:8080/sonarqube-webhook/. After adding this, the callback worked correctly and the Quality Gate resolved within seconds of analysis completion.

A second issue was network connectivity: the SonarQube server needed to be able to reach Jenkins on port 8080. My initial Security Group configuration was more restrictive than necessary and was blocking this callback. Adding an inbound rule allowing TCP on port 8080 from the SonarQube server's private IP resolved this.

15.4 Docker Permissions for Jenkins

When Jenkins attempted to run docker commands in the pipeline, it initially received a 'permission denied' error accessing /var/run/docker.sock. This is a common issue when running Docker commands as a non-root user.

The fix was adding the jenkins system user to the docker group: sudo usermod -aG docker jenkins. However, this change does not take effect for the current Jenkins session. I had to restart the Jenkins service (sudo systemctl restart jenkins) for the group membership to apply.

15.5 Nexus Docker Registry and HTTPS

Docker clients, by default, require HTTPS connections to registries. Nexus was configured to serve the Docker registry over HTTP (port 8082). When the pipeline attempted to push an image to Nexus's Docker registry, Docker rejected the connection because it was not HTTPS.

I resolved this by configuring Docker on the Jenkins server to allow insecure (HTTP) connections to the Nexus registry. I edited /etc/docker/daemon.json to add the Nexus registry IP:port to the insecure-registries array and restarted the Docker service. This is acceptable for a lab environment but in production, a proper TLS certificate should be configured for the Nexus registry.

15.6 Trivy Database Download Time

On the first run of each Trivy scan, Trivy downloads its full vulnerability database (~200 MB). This added approximately 3–4 minutes to the first pipeline run. Subsequent runs were much faster because the database was cached.

I mitigated this by running trivy image --download-db-only once on the Jenkins server as a setup step, so the database was pre-populated before the first pipeline run. I also configured Trivy to use a local cache directory (--cache-dir) that persists between pipeline builds.

15.7 SonarQube Memory Issues

As mentioned in the Results chapter, the SonarQube server's memory usage was trending upward over time on a t2.medium instance (4 GB RAM). SonarQube embeds an Elasticsearch instance that requires substantial heap memory. After several hours of continuous use, the Elasticsearch JVM heap would grow until the system ran out of memory and the SonarQube process was killed by the OS.

The solution was upgrading the SonarQube EC2 instance from t2.medium to t2.large (8 GB RAM) and configuring explicit heap size limits in sonar.properties (sonar.search.javaOpts=-Xmx2g -Xms2g). With 8 GB RAM and explicit JVM limits, the memory usage stabilized at around 5–6 GB and the OOM kills stopped.

Challenge	Root Cause	Solution
kubeadm init fails (cgroupfs)	containerd using cgroupfs instead of systemd	Set SystemdCgroup = true in containerd config
kubeadm init fails (swap)	Swap memory enabled on the node	swapoff -a and disable in /etc/fstab
kubeadm join token expired	24-hour token TTL elapsed	Regenerate token with kubeadm token create
Quality Gate hangs indefinitely	SonarQube webhook not configured to Jenkins	Add Jenkins webhook URL in SonarQube admin
Quality Gate blocked by Security Group	Jenkins port 8080 blocked from SonarQube	Add inbound rule for SonarQube IP on port 8080
Docker permission denied in pipeline	jenkins user not in docker group	usermod -aG docker jenkins; restart Jenkins
Docker refuses HTTP registry push	Docker requires HTTPS by default	Add Nexus IP:port to Docker insecure-registries
Trivy first scan very slow	Vulnerability DB download on first run	Pre-download DB with --download-db-only
SonarQube OOM crash	Elasticsearch JVM heap unbounded on low RAM	Upgrade to t2.large; set explicit JVM heap limits

CHAPTER 16: LIMITATIONS

16.1 Single Control Plane Node

The Kubernetes cluster in this project uses a single master (control plane) node. If this node fails, the cluster cannot schedule new pods and kubectl commands will not work — effectively making deployment impossible until the master is restored. For a production system, a High Availability (HA) control plane with three or more master nodes and an external etcd cluster would be required. Setting up HA kubeadm is beyond the scope of this project but is an important limitation to acknowledge.

16.2 Manual Infrastructure Provisioning

All six EC2 instances were provisioned manually through the AWS Management Console. While this worked for a single-developer project, it is not scalable or reproducible in a team environment. If I needed to recreate the environment from scratch, I would have to manually repeat all the provisioning steps. The professional solution is Infrastructure as Code (IaC) using tools like Terraform or AWS CloudFormation, which I identify as a future enhancement.

16.3 No GitOps/ArgoCD

The current pipeline uses Jenkins to directly execute kubectl commands for deployment. This imperative approach means that if someone manually modifies a Kubernetes resource in the cluster, the cluster state drifts from what the pipeline last deployed — and Jenkins has no way to detect or correct this. GitOps tools like ArgoCD solve this by continuously reconciling cluster state with the desired state in Git.

16.4 No Multi-Stage Environment

The pipeline deploys directly from build to production (or what is treated as production in this project). A real CI/CD pipeline would have separate environments:

development, staging, and production. Code would need to pass through development and staging before reaching production, giving the team more confidence in the deployment.

16.5 HTTP-Only Application and Registry

The application is served over plain HTTP (no HTTPS) and the Nexus Docker registry is configured without TLS. In production, all external endpoints should be served over HTTPS with valid TLS certificates. Setting up TLS would require either configuring certificates on the EC2 instances directly or placing the services behind an AWS Application Load Balancer with SSL termination.

16.6 No Automated Rollback

While Kubernetes performs automatic pod restart on failure, there is no automated rollback at the pipeline level. If the new deployment passes the kubectl rollout status check but the application subsequently fails (e.g., a runtime exception not caught by health checks), an operator must manually trigger a rollback using kubectl rollout undo. Automated rollback based on post-deployment monitoring metrics would be a valuable enhancement.

16.7 Single-Node Monitoring

The Prometheus and Grafana services run on a single server without redundancy. If this server fails, the monitoring visibility is completely lost. For production monitoring, a replicated Prometheus setup (using Thanos or Cortex) would ensure monitoring availability even during server failures.

CHAPTER 17: FUTURE ENHANCEMENTS

17.1 Overview

The pipeline implemented in this project represents a solid foundation, but there are many natural directions in which it can be extended and improved. The following enhancements are organized roughly in order of priority, based on the value they would add relative to the complexity of implementation.

Enhancement	Value	Effort	Priority
Terraform Infrastructure as Code	Reproducible, version-controlled infra	Medium	High
Kubernetes HA Control Plane	Eliminates master node as single point of failure	High	High
ArgoCD GitOps Deployment	Drift detection, Git-driven deployments	Medium	High
Helm Chart Deployment	Versioned releases, native rollback	Low	High
Multi-Stage Environments (dev/staging/prod)	Lower deployment risk to production	Medium	High
HTTPS/TLS for All Services	Encryption in transit	Medium	High
HashiCorp Vault for Secrets	Enterprise-grade secret management	High	Medium
ELK Stack for Log Management	Centralized log aggregation and search	Medium	Medium
Horizontal Pod Autoscaler	Automatic scale-out under load	Low	Medium
OpenTelemetry Distributed Tracing	Request tracing across services	High	Low

17.2 Terraform Infrastructure as Code

Replacing manual EC2 provisioning with Terraform scripts would make the entire infrastructure reproducible with a single terraform apply command. The Terraform

configuration would define all EC2 instances, security groups, key pairs, and networking in HCL (HashiCorp Configuration Language) files stored in the project's Git repository. This would make the infrastructure version-controlled, reviewable, and automatically consistent across deployments.

17.3 ArgoCD for GitOps

ArgoCD is a Kubernetes-native continuous delivery tool that implements the GitOps pattern. Instead of Jenkins executing kubectl commands to deploy, ArgoCD watches a Git repository for changes to Kubernetes manifests and automatically applies them to the cluster. When a new image is pushed to Docker Hub, a script updates the image tag in the manifest file and commits to Git. ArgoCD detects the commit and applies the change to the cluster. This approach provides automatic drift correction, a visual deployment history, and clean separation between CI (Jenkins) and CD (ArgoCD).

17.4 Helm Charts

Replacing raw Kubernetes YAML manifests with Helm charts would provide templating (different values for different environments), semantic versioning of deployments, and native rollback via helm rollback. The Kubernetes deployment YAML I wrote has hardcoded values (replica count, resource limits, image name) that Helm would parameterize, making the same chart usable across dev, staging, and prod with environment-specific values files.

17.5 Multi-Stage Environment Pipeline

A more mature pipeline would deploy to a development Kubernetes namespace automatically on every commit, run integration tests, promote to staging on success, and require a manual approval gate before deploying to production. Jenkins supports approval gates through the input step, and environment-specific kubeconfig

credentials can be stored in Jenkins Credentials Manager for each target cluster or namespace.

17.6 ELK Stack for Centralized Logging

The current monitoring setup (Prometheus + Grafana) provides excellent metrics visibility but no log management. An ELK stack — Elasticsearch, Logstash (or Fluent Bit), and Kibana — would aggregate logs from all Kubernetes pods and EC2 instances into a searchable, centralized log store. This would dramatically improve incident investigation speed, as operators could search logs across all services simultaneously rather than SSH-ing into individual servers.

17.7 HashiCorp Vault Integration

Jenkins Credentials Manager is adequate for a single-server setup, but for multi-team, enterprise deployments, HashiCorp Vault provides a more robust secrets management platform. Vault supports dynamic secrets (temporary credentials generated on demand and automatically expired), detailed audit logging of every secret access, and fine-grained access policies. Integrating Vault with Jenkins would replace static stored credentials with short-lived, automatically rotated credentials that significantly reduce the risk from credential exposure.

CHAPTER 18: CONCLUSION

I started this project with a clear goal: to build a real, working CI/CD pipeline — not a demo, not a simplified tutorial version, but something that genuinely demonstrates the automation and tooling that modern software teams rely on. Looking back at what was built and what was learned, I can say that goal was achieved.

The pipeline I implemented handles the complete software delivery lifecycle automatically: from a developer's code commit through compilation, security scanning, quality analysis, artifact management, containerization, orchestrated deployment, and real-time monitoring. Every one of those steps that used to require manual effort now happens without human intervention, in a consistent, auditable, and repeatable way.

The technical accomplishments are concrete: a 13-stage Jenkins declarative pipeline with average execution time under 13 minutes, a three-node Kubernetes cluster running on AWS EC2, dual-layer Trivy security scanning, SonarQube quality gates that actually block deployments on quality failures, a complete Prometheus-Grafana monitoring stack with six monitored targets. These are not theoretical — they are operational, tested, and documented with the screenshots and test results in this report.

But beyond the technical outcomes, what I value most about this project is what it taught me about real-world engineering. Textbooks describe these tools in isolation. This project required me to make them work together — and the integration challenges were where the most learning happened. The Kubernetes containerd cgroup driver issue, the SonarQube webhook configuration, the Docker group permission, the Nexus HTTP registry workaround — none of these appear in introductory tutorials, but all of them are things that someone implementing this in the real world would encounter.

The monitoring setup also gave me a practical appreciation for observability that I did not have before. When the Grafana dashboard revealed that SonarQube's memory was trending toward OOM, I was able to intervene proactively. That is the difference between reactive operations (finding out when users complain) and proactive operations (finding out before the crash). This distinction, which feels abstract in theory, became very concrete when I saw it work.

The limitations I identified — single master node, manual infrastructure, no GitOps — are honest. This project is a strong foundation, not a complete production system. The future enhancements I proposed (Terraform, ArgoCD, Helm, Vault, ELK) represent a clear roadmap for maturing this foundation into a genuinely enterprise-grade platform. I intend to pursue several of these enhancements independently after submitting this report.

For any BCA student or early-career developer considering a similar project: I strongly recommend it. The difficulty is real — expect to spend time debugging things that have nothing to do with your application code. But the learning density is also real. In the time it took to build this pipeline, I gained hands-on familiarity with AWS, Linux system administration, Docker, Kubernetes, Jenkins, SonarQube, Trivy, Nexus, Prometheus, and Grafana — a set of skills that is directly relevant to DevOps, SRE, cloud engineering, and platform engineering roles.

In conclusion: the "Automated CI/CD DevOps Pipeline for Cloud Native Application" project successfully demonstrates that a focused, methodical approach to DevOps automation — using freely available, open-source tools — can produce infrastructure that is fast, secure, observable, and genuinely useful. The work is done. The pipeline runs. The code is deployed.

REFERENCES

40. Jenkins Documentation. (2024). Jenkins User Handbook. Jenkins CI. Retrieved from https://www.jenkins.io/doc/book/
41. Apache Software Foundation. (2024). Maven: Build Lifecycle Basics. Apache Maven Project. Retrieved from https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html
42. SonarSource. (2024). SonarQube Documentation – Get Started. SonarSource. Retrieved from https://docs.sonarqube.org/latest/
43. Aqua Security. (2024). Trivy Documentation – Overview. GitHub. Retrieved from https://aquasecurity.github.io/trivy/latest/docs/
44. Sonatype. (2024). Nexus Repository Manager 3 Documentation. Sonatype. Retrieved from https://help.sonatype.com/repomanager3
45. Docker Inc. (2024). Docker Documentation. Docker Inc. Retrieved from https://docs.docker.com/
46. The Linux Foundation – CNCF. (2024). Kubernetes Documentation. Kubernetes.io. Retrieved from https://kubernetes.io/docs/home/
47. Kubernetes Community. (2024). Creating a cluster with kubeadm. Kubernetes.io. Retrieved from https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/
48. Calico Authors. (2024). Quickstart for Calico on Kubernetes. Tigera. Retrieved from https://docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart
49. Prometheus Authors. (2024). Prometheus Documentation. Prometheus.io. Retrieved from https://prometheus.io/docs/introduction/overview/
50. Grafana Labs. (2024). Grafana Documentation. Grafana Labs. Retrieved from https://grafana.com/docs/grafana/latest/
51. Amazon Web Services. (2024). Amazon EC2 Documentation. AWS. Retrieved from https://docs.aws.amazon.com/ec2/
52. Amazon Web Services. (2024). Amazon EC2 Security Groups. AWS Documentation. Retrieved from

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html

53. Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, Portland, Oregon.
54. Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations. IT Revolution Press.
55. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.
56. Dua, P., & Anand, S. (2020). Docker and Kubernetes: The Definitive Guide (3rd ed.). Packt Publishing.
57. DORA Research Team. (2023). 2023 State of DevOps Report. Google Cloud. Retrieved from https://cloud.google.com/devops/state-of-devops/
58. OWASP Foundation. (2024). OWASP Top 10: The Ten Most Critical Web Application Security Risks. OWASP. Retrieved from https://owasp.org/www-project-top-ten/
59. National Institute of Standards and Technology. (2024). National Vulnerability Database (NVD). NIST. Retrieved from https://nvd.nist.gov/

APPENDIX

Appendix A: Complete Jenkinsfile

The following is the complete declarative Jenkinsfile used in this project, as stored in the root of the application GitHub repository:

```
pipeline {
    agent any
    tools {
        jdk 'jdk17'
        maven 'maven3'
    }
    environment {
        SCANNER_HOME = tool 'sonar-scanner'
        IMAGE_NAME   = 'mdayan/cicd-app'
        NEXUS_URL    = 'http://NEXUS_IP:8081'
    }
    stages {
        stage('Git Checkout') {
            steps { git branch: 'main', credentialsId: 'github-token', url:
'https://github.com/...' }
        }
        stage('Compile')  { steps { sh 'mvn compile' } }
        stage('Test')     { steps { sh 'mvn test'    } }
        stage('Trivy File System Scan') {
            steps {
                sh 'trivy fs --format table -o trivy-fs-report.html .'
                archiveArtifacts artifacts: 'trivy-fs-report.html',
allowEmptyArchive: true
            }
        }
        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('sonar') {
                    sh '$SCANNER_HOME/bin/sonar-scanner -
Dsonar.projectName=cicd-app -Dsonar.projectKey=cicd-app -
Dsonar.java.binaries=target/classes'
                }
            }
        }
        stage('Quality Gate') {
            steps { timeout(time: 1, unit: 'HOURS') { waitForQualityGate
abortPipeline: true } }
        }
        stage('Build Artifact')  { steps { sh 'mvn package -DskipTests' } }
        stage('Publish to Nexus') {
            steps { withMaven(globalMavenSettingsConfig: 'nexus-settings',
maven: 'maven3') { sh 'mvn deploy -DskipTests' } }
        }
        stage('Docker Build') {
```

```
            steps { script { sh "docker build -t
IMAGE_NAME:BUILD_NUMBER ." } }
        }
        stage('Trivy Image Scan') {
            steps {
                sh "trivy image --format table -o trivy-image-report.html
IMAGE_NAME:BUILD_NUMBER"
                archiveArtifacts artifacts: 'trivy-image-report.html',
allowEmptyArchive: true
            }
        }
        stage('Docker Push') {
            steps { script {
                withCredentials([usernamePassword(credentialsId: 'docker-hub-
creds', usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
                    sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --
password-stdin'
                    sh "docker push IMAGE_NAME:BUILD_NUMBER"
                    sh "docker push IMAGE_NAME:latest"
                }
            }}
        }
        stage('Kubernetes Deploy') {
            steps { script {
                withCredentials([file(credentialsId: 'kubeconfig', variable:
'KUBECONFIG')]) {
                    sh 'kubectl apply -f k8s/deployment.yaml --
kubeconfig=$KUBECONFIG'
                    sh 'kubectl apply -f k8s/service.yaml --
kubeconfig=$KUBECONFIG'
                    sh 'kubectl rollout status deployment/cicd-app --
kubeconfig=$KUBECONFIG --timeout=120s'
                }
            }}
        }
        stage('Verify Deployment') {
            steps { script {
                withCredentials([file(credentialsId: 'kubeconfig', variable:
'KUBECONFIG')]) {
                    sh 'kubectl get pods --kubeconfig=$KUBECONFIG'
                    sh 'kubectl get svc  --kubeconfig=$KUBECONFIG'
                }
            }}
        }
    }
    post {
        always {
            emailext subject: "Pipeline result: BUILD_RESULT - JOB_NAME
#BUILD_NUMBER",
                     body: "Build URL: BUILD_URL",
                     to: 'team@example.com'
        }
    }
}
```

Appendix B: Kubernetes Deployment Manifest (k8s/deployment.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cicd-app
  namespace: default
  labels:
    app: cicd-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cicd-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: cicd-app
    spec:
      containers:
      - name: cicd-app
        image: mdayan/cicd-app:latest
        ports:
        - containerPort: 8080
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        readinessProbe:
          httpGet:
            path: /actuator/health
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: cicd-app
  namespace: default
spec:
  selector:
```

```
    app: cicd-app
  type: NodePort
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
    nodePort: 30000
```

Appendix C: Prometheus Configuration (prometheus.yml)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
      - targets:
          - 'JENKINS_IP:9100'
          - 'SONARQUBE_IP:9100'
          - 'NEXUS_IP:9100'
          - 'K8S_MASTER_IP:9100'
          - 'K8S_WORKER1_IP:9100'
          - 'K8S_WORKER2_IP:9100'
    relabel_configs:
      - source_labels: [__address__]
        regex: '(.*):(.*)'
        target_label: instance
        replacement: '$1'

  - job_name: 'blackbox'
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - 'http://K8S_WORKER_IP:30000'
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: 'localhost:9115'
```

Appendix D: Application Dockerfile

```
# Use slim OpenJDK 17 runtime as base
```

```
FROM openjdk:17-jre-slim

# Set working directory inside container
WORKDIR /app

# Copy the compiled JAR from the Maven build output
COPY target/*.jar app.jar

# Expose the application port
EXPOSE 8080

# Set the entry point
ENTRYPOINT ["java", "-Xmx256m", "-jar", "app.jar"]
```